



CondorNet™ Automated Trading System

Options Research Platform (CATSOR)

Technical Architecture & Mathematical Specification

Version 4.5

0. Front Matter

0.1 Document Control

Document Title

CondorNet Institutional Volatility Strategy Research Platform
Technical Architecture and Mathematical Specification

System Version

v4.5

Repository Components Covered

Component	File
Backtesting / Simulation Engine	condor_brain_backtest_v45.py
Multi-TF Data Loader & Inference Layer	backtest_v45_data.py
Strategy Template Catalog	strategies/*.py
Strategy Optimizer	strategy_optimizer.py
Strategy Defaults	strategies/_defaults.py
Strategy Loader	strategies/__init__.py

Execution Environment

Example runtime:

Component Version

Python 3.12
PyTorch 2.x
CUDA 12.x

Component Version

GPU Tesla T4
OS Linux

Primary Data Inputs

Dataset	Description
M1 dataset	high-resolution price features
M5 dataset	canonical simulation anchor
M15 dataset	intermediate regime features
H1 dataset	macro regime context
options chain	full options surface

0.2 Intended Audience

This document is written for:

Quantitative Researchers

Responsible for:

- strategy design
- volatility modeling
- statistical validation

Machine Learning Engineers

Responsible for:

- policy model training
- dataset generation
- inference integration

Execution / Market Microstructure Engineers

Responsible for:

- realistic fill modeling
- liquidity constraints
- slippage modeling

Risk / Portfolio Engineers

Responsible for:

- margin models
- capital allocation
- drawdown constraints

0.3 Scope of This System

The CondorNet platform **is not a traditional backtester**.

Instead it is a **volatility strategy research platform** designed to:

1. Enumerate **50+ multi-leg options strategies**
2. Simulate **realistic execution conditions**
3. Run **large parameter sweeps**
4. Generate **state-action trajectories**
5. Produce **training datasets for policy models**
6. Enforce **portfolio capital constraints**
7. Produce **optimization surfaces**
8. Maintain **full auditability of every decision**

The system therefore functions as a **research simulator and dataset factory** rather than a PnL estimator.

0.4 System Design Principles

The platform is designed according to the following principles.

0.4.1 Determinism

All simulation runs must be reproducible.

Given:

- identical dataset
- identical seed
- identical configuration

the system must produce **bit-identical results**.

This requires:

- deterministic ordering of strategy enumeration
- deterministic candidate ranking
- deterministic random number generation

0.4.2 Institutional Observability

Every decision made by the simulator must be explainable.

Therefore the system records:

- entry gate failures
- liquidity failures
- atomicity failures
- capital allocation failures
- parameter decoding values
- execution details

These logs enable full **post-run causal analysis**.

0.4.3 Portfolio Realism

The simulation must behave like an **actual options trading account**.

Account configuration:

$$E_0 = 100000$$

Initial equity.

Buying power with level-3 margin:

$$\begin{aligned} BP &= 2E_0 \\ BP &= 200000 \end{aligned}$$

Maximum deployable capital:

$$BP_{deploy}^{max} = 0.40 \times BP$$

$$BP_{deploy}^{max} = 80000$$

The simulator therefore must allow **multiple simultaneous positions**.

0.4.4 Separation of Concerns

The system separates responsibilities into layers.

Layer	Responsibility
Data Layer	dataset ingestion
Strategy Layer	option structure generation
Policy Layer	model inference
Execution Layer	realistic fills
Simulation Layer	lifecycle management
Optimization Layer	parameter sweeps
Dataset Layer	trajectory generation

0.4.5 Strategy Discovery

The platform is designed to evaluate a large strategy space.

11 Target strategy classes:

- Iron Condors
- Iron Butterflies
- Vertical spreads
- Short strangles
- Short straddles
- Calendars
- Diagonals
- Double diagonals
- Broken-wing spreads
- Ratio spreads
- Synthetic structures

Total templates currently implemented:

$$N_{strategies} = 58$$

Each template can produce **dozens of parameter variations**.

0.4.6 Dataset-First Philosophy

The most important artifact produced by the platform is:

state-action trajectory datasets

Each timestep produces records of the form:

$$(s_t, a_t, r_t, s_{t+1})$$

Where

Variable	Meaning
s_t	market + portfolio state
a_t	action taken
r_t	reward
s_{t+1}	next state

These datasets are used to train **CondorNet policy models**.

0.5 Reproducibility Requirements

To ensure research integrity every run must produce a **reproducibility bundle**.

The bundle includes:

Configuration Snapshot

- strategy configs
- optimizer parameters
- CLI arguments

Dataset Hash

SHA-256 hash of each input dataset.

Model Checkpoint Hash

If using inference:

hash(model_checkpoint)

Random Seed

seed

Environment Metadata

- Python version
- CUDA version
- GPU type
- OS version

0.6 Artifact Output Structure

Every simulation produces the following artifacts.

Trade Log

`reports/trades.csv`

Contains:

- OPEN events
- CLOSE events
- realized PnL
- trade lifecycle

Equity Curve

`reports/equity_curve.csv`

Contains:

equity_t

for every timestep.

Entry Gate Diagnostics

reports/bar_trace.jsonl

Records:

- gate pass/fail
 - parameter values
 - candidate ranking
-

Optimization Surfaces

reports/optimization/<strategy>/<timeframe>.csv

Each row represents a parameter combination.

1. Executive Summary

1.1 Purpose of the Platform

The CondorNet system is an **institutional volatility strategy research platform** designed to simulate, analyze, and learn from multi-leg options trading strategies.

Unlike traditional backtesting frameworks, the platform is not intended primarily to estimate historical profit and loss. Instead, it serves as a **data generation and research infrastructure** that enables systematic discovery of profitable and unprofitable options strategies and trains machine learning models to understand the conditions under which each strategy succeeds or fails.

The system performs the following core functions:

1. Enumerates a large catalog of options strategies.
2. Simulates realistic execution under market constraints.
3. Evaluates strategies across a parameter grid.
4. Generates detailed lifecycle data for every position.
5. Records decision traces explaining why trades occurred.
6. Produces datasets suitable for machine learning training.

The resulting platform supports **institutional-grade strategy discovery and policy learning**.

1.2 Evolution of the System

The project originally began as a **rule-based backtesting framework** focused primarily on iron condor strategies.

Earlier versions of the system contained the following components:

- Single-strategy simulation
- Neural signal generation
- Simplified execution modeling
- Single-position trade lifecycle

While useful for early research, this architecture was insufficient for institutional-level strategy exploration.

Version 4.5 represents a major architectural evolution. The system now functions as a **volatility strategy research engine capable of evaluating dozens of strategies simultaneously under portfolio constraints**.

Major capabilities added include:

- Strategy template catalog (58 strategies)
- Multi-dataset ingestion across multiple timeframes
- Parameter optimization loops
- Portfolio-aware capital allocation
- Execution reality modeling
- Detailed event logging
- State-action trajectory dataset generation

These capabilities transform the platform from a simple backtester into a **comprehensive research and training environment**.

1.3 Core Research Objective

The central research objective of the platform is to train a policy model capable of selecting and managing options strategies dynamically.

To achieve this, the simulator must generate large quantities of structured decision data.

Each timestep in the simulation produces a record of the form:

$$(s_t, a_t, r_t, s_{t+1})$$

Where

Variable	Meaning
s_t	market and portfolio state
a_t	action taken
r_t	reward observed
s_{t+1}	next state

This structure enables training methods drawn from:

- reinforcement learning
- policy optimization
- imitation learning
- supervised classification

The simulator therefore functions as a **trajectory dataset generator**.

1.4 Strategy Exploration Space

The system explores a large space of options strategies.

Currently implemented strategy templates include:

- single-leg option selling
- vertical credit spreads
- vertical debit spreads
- iron condors
- iron butterflies
- short strangles
- short straddles
- calendars
- diagonals
- double diagonals
- broken-wing spreads
- ratio spreads
- synthetic structures

Total number of strategy templates currently available:

$$N_{strategies} = 58$$

Each strategy may be evaluated across a parameter grid including:

- days to expiration
- strike offsets
- spread widths

- profit targets
- stop loss policies
- contract sizing

The resulting search space contains **thousands of possible strategy configurations**.

1.5 Portfolio Simulation Model

The simulator operates under a realistic portfolio model.

Initial account equity is defined as:

$$E_0 = 100000$$

The account is assumed to have options trading level 3 permissions and 2:1 margin leverage.

Total buying power is therefore:

$$\begin{aligned} BP &= 2E_0 \\ BP &= 200000 \end{aligned}$$

However, to maintain conservative portfolio management, only a fraction of buying power may be deployed into options strategies.

The maximum deployable capital is defined as:

$$\begin{aligned} BP_{deploy}^{max} &= 0.40 \times BP \\ BP_{deploy}^{max} &= 80000 \end{aligned}$$

The simulator therefore manages a **portfolio of concurrent positions**, allocating capital dynamically across strategies while ensuring that the total collateral used never exceeds the defined limit.

1.6 Execution Reality Modeling

Accurate strategy evaluation requires realistic modeling of options market microstructure.

The simulator incorporates a simplified execution engine that models:

- bid-ask spreads
- mid-price execution
- slippage

- multi-leg atomic fills
- latency effects

The mid price of an option is defined as

$$mid = \frac{bid + ask}{2}$$

The spread percentage is defined as

$$spread\% = \frac{ask - bid}{mid}$$

Trades that exceed defined liquidity thresholds are rejected to prevent unrealistic fills.

1.7 Optimization Engine

The system includes a strategy optimization engine that evaluates parameter combinations across historical data.

For each strategy template, the optimizer sweeps parameters such as:

- stop loss thresholds
- profit targets
- position limits
- cooldown periods
- contract sizing

Each parameter combination produces a result row containing metrics such as:

- total return
- maximum drawdown
- win rate
- average holding time
- capital efficiency

The optimizer ranks results according to configurable objective functions, allowing the researcher to identify promising strategy configurations.

1.8 Research Output Artifacts

Each simulation produces several artifacts used for analysis and model training.

Trade lifecycle log:

`reports/trades.csv`

Equity curve:

`reports/equity_curve.csv`

Entry decision trace:

`reports/bar_trace.jsonl`

Optimization surfaces:

`reports/optimization/<strategy>/<timeframe>.csv`

Trajectory datasets used for training:

`datasets/trajectory_dataset.parquet`

These artifacts enable both **quantitative evaluation and machine learning training workflows**.

1.9 System Importance

The CondorNet platform enables systematic study of volatility strategies that would otherwise be impossible to evaluate manually.

By simulating thousands of strategies across a full year of market data and recording every decision made during the simulation, the system provides a rich dataset for training models capable of:

- identifying favorable volatility regimes
- selecting optimal strategies
- adjusting positions dynamically
- avoiding unfavorable conditions

The platform therefore represents a **foundation for automated options portfolio management**.

2. Codebase Map and How Everything Connects

This chapter describes the internal architecture of the CondorNet v4.5 research platform and explains how the codebase components interact during execution.

The system is composed of four primary layers:

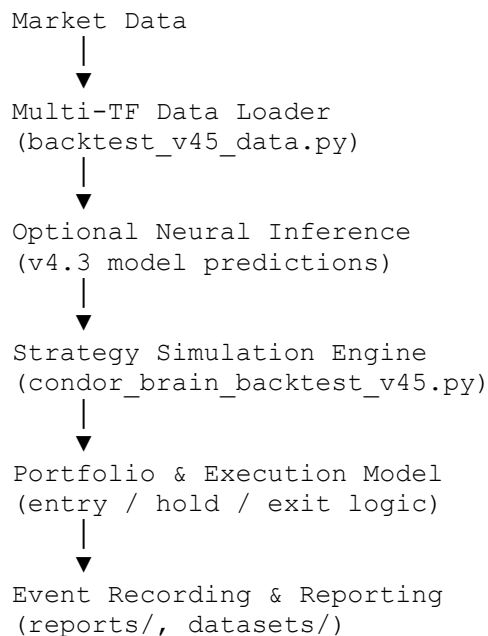
1. **Data Layer** — ingestion and alignment of multi-timeframe datasets
2. **Inference Layer** — optional neural signal generation (v4.3 model)
3. **Simulation Layer** — portfolio lifecycle simulation and execution modeling
4. **Recording Layer** — event logs, metrics, and dataset generation

The architecture is designed so that **each layer can evolve independently**, allowing new strategies, models, and datasets to be integrated without restructuring the simulator core.

2.1 High-Level Dataflow Diagram

The runtime system processes market data through a sequence of stages that ultimately produce both **trading performance metrics** and **state-action datasets for machine learning**.

The high-level dataflow can be summarized as:



Conceptually, the runtime pipeline follows the transformation:

Data → Inference → Simulation → Recording

Where

Stage	Purpose
Data	ingest and align market datasets
Inference	compute model signals used by strategies
Simulation	evaluate strategies under portfolio constraints
Recording	produce logs and training datasets

The simulator therefore acts as both:

1. **a research backtesting engine**
2. **a trajectory dataset generator**

2.2 Runtime Entrypoints

The CondorNet platform provides two primary runtime entrypoints depending on the research task.

Primary Backtesting Entrypoint

The standard execution path begins in:

```
condor_brain_backtest_v45.py
```

The main runtime function is:

```
main()
```

This function performs the following steps:

1. Parse command-line arguments
2. Load datasets
3. Optionally load a neural model
4. Initialize strategy templates
5. Run the simulation loop
6. Generate output artifacts

Conceptually:

`main() → run_backtest()`

Optimization Entrypoint

Strategy optimization is executed via:

```
strategy_optimizer.py
```

The core driver function is:

```
run_optimizer(...)
```

This module runs multiple backtests with varying parameter configurations in order to identify optimal strategy settings.

The optimizer therefore acts as an **external loop controller** around the core simulation engine.

Conceptually:

Optimizer → Repeated Backtests

2.3 Module Responsibilities

The CondorNet codebase is organized around several key modules, each responsible for a distinct layer of the system.

condor_brain_backtest_v45.py

Primary role: **simulation engine**

This module implements the core portfolio simulation environment.

Major responsibilities include:

- simulation loop
- portfolio state management
- entry and exit decision logic
- multi-leg option construction
- capital allocation constraints
- execution realism modeling
- trade lifecycle tracking
- event logging and reporting

The engine supports **multiple simultaneous positions per strategy**, enabling a true portfolio simulation rather than a single-position backtest.

backtest_v45_data.py

Primary role: **data ingestion and preprocessing**

This module defines the `MultiTFDataBundle` abstraction, which organizes market datasets across multiple timeframes.

Responsibilities include:

- loading M1, M5, M15, and H1 datasets
- aligning timestamps across timeframes
- applying feature normalization
- constructing model inference batches
- exposing synchronized market state during simulation

The module ensures that the simulator operates on a **single canonical time axis**, typically the M5 dataset.

strategies/

Primary role: **strategy template catalog**

The `strategies/` directory contains a collection of strategy template definitions.

Each template defines a `CONFIG` structure that describes:

- strategy type
- leg structure
- strike selection rules
- DTE targeting policy
- stop loss behavior
- profit targets
- margin model

The platform currently includes **58 strategy templates**, spanning:

- single-leg option selling
- vertical spreads

- condors
- butterflies
- straddles and strangles
- calendars and diagonals
- broken-wing spreads
- ratio spreads
- synthetic option structures

Strategy defaults are defined in:

strategies/_defaults.py

The 58 templates:

/spy-iron-condor-trading/kaggle/strategies tree

```
— bear_call_ladder.py
— bear_call_spread_credit.py
— bear_put_ladder.py
— bear_put_spread.py
— bull_call_ladder.py
— bull_call_spread.py
— bull_put_ladder.py
— bull_put_spread_credit.py
— calendar_call.py
— calendar_put.py
— call_broken_wing.py
— call_ratio_backspread.py
— call_ratio_spread.py
— cash_secured_put.py
— collar.py
— covered_call.py
— covered_short_straddle.py
— covered_short_strangle.py
— diagonal_call.py
— diagonal_put.py
— double_diagonal.py
— guts.py
— inverse_call_broken_wing.py
— inverse_iron_butterfly.py
— inverse_iron_condor.py
— inverse_put_broken_wing.py
— iron_butterfly.py
— iron_condor.py
— jade_lizard.py
— long_call.py
— long_call_butterfly.py
— long_call_condor.py
— long_combo.py
— long_put.py
— long_put_butterfly.py
— long_put_condor.py
— long_synthetic_future.py
— protective_put.py
— put_broken_wing.py
```

```
|— put_ratio_backspread.py
|— put_ratio_spread.py
|— reverse_jade_lizard.py
|— short_call.py (done)
|— short_call_butterfly.py
|— short_call_condor.py
|— short_combo.py
|— short_guts.py
|— short_put.py
|— short_put_butterfly.py
|— short_put_condor.py
|— short_straddle.py
|— short_strangle.py
|— short_synthetic_future.py
|— straddle_long.py
|— strangle_long.py
|— strap.py
|— strip.py
|— synthetic_put.py
```

Templates inherit these values through a configuration merge process.

The loader logic resides in:

```
strategies/__init__.py
```

which dynamically discovers strategy files and builds the runtime template catalog.

strategy_optimizer.py

Primary role: **parameter optimization**

The optimizer performs systematic parameter sweeps across the strategy catalog.

Typical parameters explored include:

- stop loss thresholds
- profit targets
- position limits
- cooldown periods
- contract sizing rules

Each configuration is evaluated by running a full simulation.

Results are ranked using objective functions such as:

$$J(\theta) = \alpha_1 \cdot \text{Return} - \alpha_2 \cdot \text{Drawdown} + \alpha_3 \cdot \text{Sharpe}$$

The optimizer produces ranked surfaces identifying promising strategy configurations.

2.4 Call Graph Overview

This section summarizes how the main modules interact during execution.

Standard Backtest Flow

The primary execution path is:

```
main()
├── load_v43_model()           (optional)
├── load_multi_tf_bundle()
├── run_v43_batch_inference()  (optional)
└── run_backtest()
```

Where:

Function	Purpose
load_v43_model()	loads neural inference model
load_multi_tf_bundle()	loads aligned datasets
run_v43_batch_inference()	generates signal features
run_backtest()	executes the simulation

The `run_backtest()` routine implements the full portfolio lifecycle:

```
for each bar:
    update portfolio state
    evaluate exits
    evaluate entries
    apply capital constraints
    record events
```

Optimizer Execution Flow

The optimizer runs the simulator repeatedly under different parameter settings.

Conceptually:

```
run_optimizer()  
├── generate parameter grid  
├── for each parameter set:  
│   └── run_backtest()  
└── rank results
```

This produces a parameter-performance surface for each strategy template.

2.5 Required Runtime Assets

Successful execution of the CondorNet platform requires several external assets.

Market Datasets

The simulator expects preprocessed datasets for multiple timeframes.

Typical directory structure:

```
data/  
  Datasetv4/  
    v43/  
      m1_dataset_v43_final.csv  
      m5_dataset_v43_final.csv  
      m15_dataset_v43_final.csv  
      h1_dataset_v43_final.csv
```

These datasets provide the core market features used by the simulator.

Options Chain Data

The simulation also requires historical options chain data.

This data is used to construct strategy legs during trade entry.

Typical structure:

```
options_data/  
  SPY/  
    YYYY-MM-DD/  
      chain.csv
```

Each simulation bar accesses the corresponding chain slice.

Model Checkpoints (Optional)

When neural signals are enabled, the simulator loads a trained model.

Example location:

```
models/  
    condornet_v43_checkpoint.pt
```

This model generates predictive features used by strategy entry rules.

Output Artifacts

Simulation results are written to the reports directory.

Example structure:

```
reports/  
    trades.csv  
    equity_curve.csv  
    bar_trace.jsonl
```

Optimizer runs generate additional artifacts:

```
reports/  
    optimization/  
        <strategy_name>/  
            results.csv
```

Trajectory datasets for machine learning training may also be generated:

```
datasets/  
    trajectories/  
        <run_id>.parquet
```

These files contain state-action sequences suitable for reinforcement learning or supervised training pipelines.

3. Data Architecture (v4.5 / Phase 7)

The CondorNet v4.5 simulator operates on a structured multi-timeframe dataset stack designed to represent market conditions at several temporal resolutions simultaneously.

The purpose of the data architecture is to ensure that:

1. All features are **time-synchronized**
2. Model inference receives **consistent feature vectors**
3. The simulator operates on a **single canonical clock**
4. Multi-resolution market context is preserved

The data system is implemented primarily within:

```
backtest_v45_data.py
```

which defines the **MultiTFDataBundle** abstraction used throughout the simulator.

3.1 Dataset Stack and Canonical Clock

The CondorNet simulator ingests multiple

datasets corresponding to different market timeframes.

The current dataset stack includes:

Dataset	Description
M1	high-resolution price features
M5	canonical simulation clock
M15	intermediate regime features
H1	macro regime context

Each dataset contains engineered features derived from the underlying SPY price series.

However, the simulation loop itself operates on a **single canonical time index**.

For v4.5, the canonical clock is defined as:

$$T = \{t_1, t_2, \dots, t_N\}$$

where

T = timestamps of the M5 dataset

Thus the simulation advances according to the M5 bar sequence.

At each canonical timestep t_k , the simulator retrieves the corresponding observations from the other timeframes.

Formally:

$$x_k = (x_k^{M1}, x_k^{M5}, x_k^{M15}, x_k^{H1})$$

where

$$x_k^{TF}$$

denotes the feature vector from timeframe TF aligned to canonical time t_k .

3.2 MultiTFDataBundle Specification (backtest_v45_data.py)

The MultiTFDataBundle object provides a structured container for the aligned datasets.

It encapsulates both the raw data arrays and the timestamp alignment mappings.

Conceptually, the bundle contains:

```
MultiTFDataBundle
├── timestamps_m1
├── timestamps_m5
├── timestamps_m15
├── timestamps_h1
├── data_m1
├── data_m5
├── data_m15
├── data_h1
├── feature_stats
└── alignment_maps
```

The object maintains the following invariants:

1. The M5 dataset defines the canonical time axis.
2. All other datasets must be alignable to the M5 timestamps.
3. Feature matrices must contain identical column ordering across runs.

Typical shapes of the dataset tensors are:

Dataset	Shape
M1	$N_{M1} \times F$
M5	$N_{M5} \times F$
M15	$N_{M15} \times F$
H1	$N_{H1} \times F$

Where

F = number of engineered features

During simulation, the aligned feature vector returned for each bar is therefore:

$$x_t \in \mathbb{R}^{4F}$$

constructed by concatenating the features from each timeframe.

3.3 Alignment Mathematics

Because the datasets contain different sampling intervals, timestamps must be aligned to the canonical M5 clock.

For each canonical timestamp t_k , the simulator must locate the most recent observation in the other timeframes.

Let

$$T^{M5} = \{t_1^{M5}, \dots, t_N^{M5}\}$$

and

$$T^{M15} = \{t_1^{M15}, \dots, t_M^{M15}\}$$

The alignment rule is defined as:

$$i = \max \{j \mid t_j^{TF} \leq t_k^{M5}\}$$

which selects the most recent timestamp not exceeding the canonical time.

In implementation, this mapping is computed using a binary search operation:

```
idx = searchsorted(other_ts, m5_ts, side='right') - 1
```

Where:

Variable	Meaning
other_ts	timestamp array of the target timeframe
m5_ts	canonical M5 timestamp
idx	aligned index

This operation is equivalent to computing:

$$idx = \arg \max_j \{t_j^{TF} \leq t_k^{M5}\}$$

Edge Behavior

Two edge cases must be handled.

Case 1 — Pre-start alignment

If

$$t_1^{TF} > t_k^{M5}$$

then the dataset does not yet contain an observation.

The alignment index is set to:

$$idx = 0$$

or the row may be treated as missing depending on configuration.

Case 2 — End-of-dataset alignment

If

$$t_k^{M5} > t_M^{TF}$$

then the final observation of the timeframe is reused.

$$idx = M$$

This ensures the simulator remains defined across the entire canonical clock.

3.4 Feature Normalization Mathematics

To stabilize machine learning training and reduce sensitivity to scale differences, features are normalized using a **robust z-score transformation**.

Unlike standard normalization based on mean and standard deviation, the system uses the **median and median absolute deviation (MAD)**.

Fit Step

Given a feature vector

$$x = (x_1, x_2, \dots, x_n)$$

the median is defined as

$$m = \text{median}(x)$$

The median absolute deviation is

$$MAD = \text{median}(|x_i - m|)$$

Consistency Scaling

To make the MAD estimator consistent with the standard deviation for normally distributed data, a scaling constant is applied.

$$\sigma_{robust} = 1.4826 \cdot MAD$$

Transformation

Each feature is transformed using

$$z = \frac{x - m}{1.4826 \cdot MAD}$$

This produces a robust standardized variable.

Clipping

To prevent extreme outliers from destabilizing the model, normalized values are clipped:

$$z_{clipped} = \max(\min(z, z_{max}), z_{min})$$

Typical bounds are

$$\begin{aligned} z_{min} &= -10 \\ z_{max} &= 10 \end{aligned}$$

Clipping prevents rare anomalies from dominating model gradients.

3.5 Options Chain Ingestion

In addition to market feature datasets, the simulator requires historical options chain data in order to construct strategy legs.

The chain ingestion system retrieves option quotes corresponding to each simulation bar.

Date Partitioning

Options chains are typically stored by trading day.

Example directory structure:

```
options_data/  
  SPY/  
    2025-01-02/  
      chain.csv
```

This structure allows efficient loading of the relevant chain file for each simulation day.

Per-Bar Chain Slice Retrieval

At each simulation timestep t_k , the simulator retrieves the option quotes corresponding to that timestamp.

Let

$$C_d$$

be the chain dataset for trading day d .

The simulator selects all rows satisfying

$$timestamp = t_k$$

producing the bar-specific chain slice:

$$C_{t_k}$$

This slice contains all option contracts available at that moment.

Chain Grid Construction

Two different representations of the chain are used.

1. Inference Grid

A structured matrix representation used by neural models.

This grid typically contains features such as:

- implied volatility
 - option Greeks
 - bid/ask prices
 - strike offsets
-

2. Raw Chain Representation

The simulator retains the raw chain rows for strategy construction.

This raw representation is used to:

- locate strikes matching template criteria
- compute credit/debit values
- validate liquidity constraints
- construct multi-leg spreads

3.6 Missing Data Policy

Financial datasets frequently contain missing observations due to market conditions, feature generation failures, or dataset boundaries.

The CondorNet system implements several safeguards to prevent missing data from corrupting simulation results.

NaN Handling

Numeric features containing NaN values are handled according to feature type.

Typical policies include:

- forward fill from previous observation
- replacement with feature median
- exclusion of affected rows

Pivot Masks

Some feature sets contain sparse indicators such as pivot levels.

These are represented using **pivot masks** that identify whether the value is valid.

For a feature p_t , the mask is defined as

$$m_t = \begin{cases} 1 & \text{if pivot exists} \\ 0 & \text{otherwise} \end{cases}$$

Models therefore receive both:

- pivot magnitude
- pivot validity indicator

Chain Integrity Gates

Option chains may occasionally contain incomplete data.

Before a strategy can be opened, the simulator verifies that the chain satisfies integrity conditions.

Typical checks include:

- bid and ask prices present
- positive liquidity
- valid strike ordering
- non-zero spreads

If any required contract data is missing, the entry attempt fails with a **chain integrity gate rejection**.

This ensures that simulated trades only occur under conditions where the strategy could realistically be executed in live markets.

4. Strategy Catalog System (Templates → Executable Structures)

The CondorNet platform evaluates a large catalog of options trading strategies defined as **parameterized templates**.

Rather than hardcoding individual trading rules, the system defines each strategy as a **configuration object** describing:

- leg structure
- strike targeting logic
- expiration selection
- entry constraints
- risk management policies

These templates are stored as Python modules in:

```
strategies/
```

Each module defines a `CONFIG` dictionary describing the strategy.

During runtime the simulator loads these templates, merges them with global defaults, and converts them into **executable trade construction instructions**.

The catalog therefore acts as the **strategy definition layer** of the platform.

4.1 Strategy Taxonomy and Naming Standard

Strategy templates follow a naming convention reflecting the underlying option structure.

Template names correspond directly to the Python module filename.

Example:

```
short_call.py  
iron_condor.py  
bull_put_spread_credit.py
```

The naming convention follows the general form:

<direction>_<structure>_<modifier>

Examples:

Template	Description
short_call	single-leg call selling
short_put	single-leg put selling
bull_put_spread_credit	credit spread using puts
bear_call_spread_credit	credit spread using calls
iron_condor	short call spread + short put spread
long_call_butterfly	symmetric call butterfly
double_diagonal	multi-expiration diagonal structure

This convention allows researchers to quickly infer the underlying structure of the strategy.

4.2 Template Configuration Model (CONFIG Contract)

Each strategy template defines a dictionary named CONFIG.

Example structure:

```
CONFIG = {  
    "strategy_name": "short_call",  
    "structure": "single_leg",  
    "option_type": "call",  
    "side": "sell",  
    "legs": [...],  
    "entry_rules": {...},  
    "exit_rules": {...},  
    "margin_model": "...",  
}
```

The `CONFIG` dictionary functions as a **declarative specification** describing how the strategy should be constructed.

Typical fields include:

Key	Meaning
<code>strategy_name</code>	unique identifier
<code>structure</code>	structural category
<code>legs</code>	list describing option legs
<code>entry_rules</code>	entry constraints
<code>exit_rules</code>	profit / stop policies
<code>margin_model</code>	collateral calculation method
<code>max_positions</code>	maximum simultaneous positions

The simulator interprets this configuration at runtime and converts it into **actual option trades**.

4.3 Global Defaults (`strategies/_defaults.py`)

Many strategy parameters are shared across templates.

These shared values are defined in:

`strategies/_defaults.py`

The defaults file provides baseline values for configuration fields.

Example categories include:

- default profit targets
- stop loss multipliers
- contract sizing
- cooldown periods
- maximum concurrent positions

Inheritance / Merge Rules

When a strategy template is loaded, the configuration dictionary is merged with the defaults.

Conceptually:

$$CONFIG_{final} = merge(CONFIG_{defaults}, CONFIG_{template})$$

Where template values override defaults.

Example:

```
defaults:
    profit_target = 0.5

template override:
    profit_target = 0.3
```

Result:

```
profit_target = 0.3
```

This mechanism allows the platform to maintain consistent policy behavior while permitting template-specific overrides.

Key Meanings and Allowed Ranges

Several configuration parameters must satisfy predefined ranges to ensure realistic trading behavior.

Examples include:

Parameter	Typical Range
profit_target	0.1 – 1.0 (fraction of credit)
stop_loss_mult	1.0 – 5.0
max_positions	1 – 50
cooldownBars	≥ 0
max_contracts	positive integer

These constraints are enforced during template validation.

4.4 Template Loader Behavior

Strategy templates are dynamically discovered during runtime.

The loader logic resides in:

```
strategies/__init__.py
```

Discovery Rules

The loader scans the strategies directory for Python modules.

Conceptually:

```
strategies/  
    *.py
```

Each file is imported and inspected for a `CONFIG` object.

If present, the template is registered in the strategy catalog.

Exclusion Rules

Files beginning with an underscore are ignored.

Example:

```
_defaults.py
```

This prevents internal modules from being interpreted as strategies.

The exclusion rule can be expressed as:

$$filename \notin \{f: f[0] = "_"\}$$

Validation Rules

Before a template is accepted, the loader verifies that required keys exist.

Typical required keys include:

- `strategy_name`
- `legs`
- `structure`
- `entry_rules`
- `exit_rules`

If any required field is missing, the template fails validation and is excluded from the catalog.

This ensures that all strategies satisfy the minimum structural requirements required by the simulator.

4.5 Template → Legs Mapping

The most important component of a strategy template is the **leg specification**.

This defines how individual option contracts are selected.

Each strategy contains one or more legs.

Leg Schema

Each leg is defined by a schema containing the following fields:

Field	Meaning
type	call or put
side	buy or sell
qty	number of contracts
rank	strike targeting rank
dte_target	desired days-to-expiration

Example leg definition:

```
{  
  "type": "call",  
  "side": "sell",  
  "qty": 1,  
  "rank": 0,  
  "dte_target": 45  
}
```

Rank Semantics

The `rank` parameter determines how the strike is selected relative to the underlying price.

Examples include:

Rank	Meaning
0	ATM strike
+1	next OTM strike

+2	second OTM strike
-1	ITM strike

Rank-based selection allows strategies to adapt dynamically to the available option chain.

Multi-Leg Construction

Strategies with multiple legs define a list of leg objects.

Example — Iron Condor:

```
SELL put spread  
SELL call spread
```

Leg representation:

```
[  
    short_put,  
    long_put,  
    short_call,  
    long_call  
]
```

The simulator constructs the full position by locating contracts matching these specifications in the option chain.

4.6 Current Catalog Inventory (58 Templates)

The current strategy catalog contains **58 templates**.

These templates span several structural categories.

A) Single-Leg Premium Strategies

Strategies that involve selling or buying a single option.

Examples:

- short_call
- short_put
- long_call

- long_put
- cash_secured_put

These strategies have the simplest structure but often carry directional risk.

B) Vertical Spreads (Credit / Debit)

Vertical spreads involve two options with the same expiration but different strikes.

Examples:

- bull_put_spread_credit
- bear_call_spread_credit
- bull_call_spread
- bear_put_spread

These strategies cap both maximum profit and maximum loss.

C) Condors and Butterflies

These strategies involve four option legs forming symmetric structures.

Examples:

- iron_condor
- iron_butterfly
- long_call_butterfly
- short_call_condor

They are commonly used for volatility trading.

D) Calendars and Diagonals

These strategies involve options with different expiration dates.

Examples:

- calendar_call
- calendar_put

- diagonal_call
- diagonal_put
- double_diagonal

These strategies attempt to exploit differences in time decay.

E) Collars, Synthetics, and Combos

More complex strategies combining multiple option positions.

Examples:

- collar
- synthetic_put
- synthetic_future
- long_combo
- short_combo

These structures can replicate the payoff of other financial instruments.

5. Simulation Engine (condor_brain_backtest_v45.py)

The CondorNet simulation engine is implemented in:

`condor_brain_backtest_v45.py`

This module is responsible for executing the full lifecycle of strategy evaluation across historical market data.

The simulation engine performs the following core functions:

- portfolio state management
- option position construction
- execution modeling
- strategy evaluation
- capital constraint enforcement
- trade lifecycle tracking
- dataset generation

Unlike traditional backtesters that simulate a single strategy, the CondorNet engine supports **simultaneous evaluation of multiple strategies within a shared portfolio environment.**

At any time t , the simulator maintains a portfolio:

$$\mathcal{P}_t = \{p_1, p_2, \dots, p_{N_t}\}$$

where

$$N_t = |\mathcal{P}_t|$$

is the number of active positions.

5.1 Simulation Modes

The simulation engine supports multiple runtime modes depending on the research configuration.

Legacy Mode (M1-Centric / v2.2 Style)

Legacy mode corresponds to the architecture used in earlier versions of the platform.

Characteristics include:

- simulation driven by the **M1 dataset**
- simplified market state construction
- rule-based strategy evaluation
- limited integration with neural inference

In this mode, the canonical time axis is defined as:

$$T = \{t_1^{M1}, t_2^{M1}, \dots, t_N^{M1}\}$$

Although still supported for compatibility, this mode is primarily used for regression testing and historical comparisons.

v4.3 Inference Mode (`--use-v43`)

The modern simulation mode integrates predictions from the CondorNet v4.3 neural model.

When the `--use-v43` flag is enabled:

- the **M5 dataset becomes the canonical simulation clock**
- model predictions are generated for each bar
- strategies may incorporate model outputs as entry signals

The canonical time axis becomes:

$$T = \{t_1^{M5}, t_2^{M5}, \dots, t_N^{M5}\}$$

At each timestep t_k , the system constructs a market state vector:

$$x_t = (x_t^{M1}, x_t^{M5}, x_t^{M15}, x_t^{H1}, \hat{y}_t)$$

where

$$\hat{y}_t$$

represents model predictions generated by the v4.3 network.

This architecture enables the platform to evaluate strategies conditioned on machine learning signals.

5.2 Bar Loop Lifecycle

The simulation proceeds through a sequential loop over the canonical time axis.

For each bar t_k , the engine performs a series of operations.

Conceptually, the loop follows:

```
for each bar t_k:  
    build market state  
    compute marks  
    update open positions  
    evaluate exits  
    evaluate entries  
    enforce capital constraints  
    record events
```

Step 1 — Build Per-Bar Market State

The simulator constructs the market state vector:

$$x_t$$

which includes:

- multi-timeframe price features
- neural model predictions (optional)
- current portfolio summary

The portfolio state function can be represented as:

$$\phi(\mathcal{P}_t)$$

Thus the full simulator state is:

$$s_t = (x_t, \phi(\mathcal{P}_t))$$

Step 2 — Compute Marks

Each open position must be marked to market.

For an option contract with bid price P_{bid} and ask price P_{ask} , the mark price is:

$$P_{mid} = \frac{P_{bid} + P_{ask}}{2}$$

For a multi-leg position containing contracts $i = 1 \dots k$:

$$V_{position} = \sum_{i=1}^k q_i P_{mid,i}$$

where

$$q_i$$

is the signed contract quantity.

This produces the current position valuation.

Step 3 — Update Open Positions

For each open position $p_i \in \mathcal{P}_t$, the simulator updates:

- unrealized profit and loss
- days to expiration

- maximum adverse excursion
- high-water mark

Position PnL is defined as:

$$PnL_t = V_{entry} - V_{current}$$

for credit positions.

Step 4 — Evaluate Exit Conditions

Exit rules are evaluated for each position.

Common exit conditions include:

- profit target reached
- stop loss triggered
- expiration threshold reached
- liquidity failure

Formally, an exit occurs when a predicate evaluates to true:

$$Exit(p_i, s_t) = True$$

If triggered, the position is closed and removed from the portfolio:

$$\mathcal{P}_{t+1} = \mathcal{P}_t \setminus \{p_i\}$$

Step 5 — Evaluate Entry Conditions

After exits are processed, the engine evaluates new entry opportunities.

For each strategy template $s \in S$, the system checks:

$$Entry(s, s_t)$$

If the entry signal passes and sufficient capital exists, a new position is constructed.

Step 6 — Enforce Capital Constraints

Before opening a new position, the simulator verifies that the required margin does not exceed the portfolio deployment limit.

Let

$$C_{deploy,t} = \sum_{p_i \in \mathcal{P}_t} M_i$$

where M_i is the margin requirement of position i .

The capital constraint is:

$$C_{deploy,t} + M_{new} \leq BP_{deploy}^{max}$$

If the constraint fails, the trade is rejected.

Step 7 — Record Events and Trajectories

All significant events are recorded for later analysis.

Examples include:

- OPEN events
- CLOSE events
- HOLD states
- rejected entries

These records are written to:

`reports/bar_trace.jsonl`

Additionally, trajectory datasets may be generated for machine learning:

$$(s_t, a_t, r_t, s_{t+1})$$

5.3 Trade Object Model

The simulator represents each position using a structured trade object.

Conceptually:

$$p_i = (s_i, \theta_i, L_i, t_{entry}, C_i, M_i)$$

Where:

Variable	Meaning
s_i	strategy template
θ_i	parameter configuration
L_i	set of option legs
t_{entry}	entry timestamp
C_i	credit or debit
M_i	margin requirement

Trade Schema

The trade object typically includes the following fields:

Field	Meaning
trade_id	unique identifier
strategy	template name
entry_time	timestamp of entry
entry_credit	premium received
max_loss	worst-case loss
contracts	number of contracts
legs	option leg objects
dte_entry	days-to-expiration at entry
dte_remaining	remaining DTE
unrealized_pnl	current PnL
max_adverse_excursion	largest drawdown
high_water_mark	highest unrealized profit

Invariants

Several invariants must always hold for trade objects.

Position Consistency

The number of legs must match the strategy definition.

$$|L_i| = L_{strategy}$$

Margin Validity

Margin must be non-negative:

$$M_i \geq 0$$

Unique Trade IDs

Each position must have a unique identifier:

$$trade_id_i \neq trade_id_j$$

for

$$i \neq j$$

6. Execution Reality Modeling (Phase 5.2 + 5.5)

The CondorNet simulator incorporates an execution reality layer that attempts to approximate how options trades are filled in real markets.

This component exists to prevent unrealistic results produced by simplified backtesting approaches that assume fills at the close price or midpoint without accounting for bid–ask spreads, liquidity constraints, or slippage.

Execution reality modeling ensures that simulated trades reflect conditions closer to those encountered during live trading.

The execution model therefore acts as an intermediate layer between:

Strategy Decision Engin

and

Trade Lifecycle System

It determines whether a trade:

- can be filled,
 - at what price it is filled,
 - and under what conditions it may fail.
-

6.1 Why Execution Reality Exists

Many traditional backtesting frameworks assume that trades are executed at the closing price of the underlying asset or option contract.

This assumption leads to unrealistic performance estimates because it ignores several critical aspects of market microstructure:

- bid–ask spreads
- liquidity limitations
- multi-leg fill complexity
- transaction costs
- slippage

Such simplified models produce what is often referred to as **close-based fantasy pricing**, where trades are assumed to execute at idealized prices that may not actually exist in the order book.

The CondorNet platform therefore introduces a structured execution model that approximates real-world fills using observed option quotes.

This approach improves the alignment between simulated performance and live trading behavior.

6.2 Bid–Ask–Mid Synthesis Model

Each option contract in the chain provides two primary observable prices:

$$P_{bid} \ P_{ask}$$

The bid price represents the highest price buyers are currently willing to pay, while the ask price represents the lowest price sellers are willing to accept.

Mid Price Definition

The midpoint price is defined as:

$$P_{mid} = \frac{P_{bid} + P_{ask}}{2}$$

This value provides a neutral estimate of the contract's fair value between buyers and sellers.

Spread Model

The absolute bid–ask spread is defined as:

$$S = P_{ask} - P_{bid}$$

The relative spread percentage is defined as:

$$S_{\%} = \frac{P_{ask} - P_{bid}}{P_{mid}}$$

Spreads represent the cost of crossing the market and are therefore a primary source of execution friction.

Mark Model

For valuation purposes, the simulator marks option positions to the midpoint price.

For a contract with quantity q :

$$V_{contract} = q \cdot P_{mid}$$

For a multi-leg position containing k contracts:

$$V_{position} = \sum_{i=1}^k q_i P_{mid,i}$$

This valuation provides a consistent estimate of the position's unrealized value during the simulation.

Slippage Model

Slippage represents the difference between the theoretical execution price and the actual price obtained in the market.

The simulator models slippage as a function of the spread.

For a trade attempting to cross the spread, the effective fill price may be expressed as:

$$P_{fill} = P_{mid} + \lambda S$$

where

$$\lambda \in [-0.5, 0.5]$$

controls the position of the fill within the spread.

Typical assumptions include:

Scenario	λ value
ideal midpoint fill	0
marketable order	0.5
favorable execution	-0.25

These assumptions approximate the execution behavior of limit orders and market orders.

6.3 Entry Fill Model

When a strategy attempts to open a position, the simulator must determine whether the order can be filled.

This process involves several checks.

Atomicity Definition

Multi-leg strategies must be executed as a single atomic transaction.

Let

$$L = \{\ell_1, \ell_2, \dots, \ell_k\}$$

be the set of legs for a strategy.

The entry order is considered successful only if **all legs can be filled simultaneously**.

Formally:

$$Fill(L) = \begin{cases} 1 & \text{if all legs fill} \\ 0 & \text{otherwise} \end{cases}$$

If any leg fails to meet liquidity or pricing requirements, the entire trade is rejected.

Partial Fill Rules

The CondorNet simulator does **not allow partial fills** for multi-leg strategies.

Partial execution would leave the portfolio exposed to unintended directional risk.

Therefore the rule is:

$$PartialFill = False$$

Any failure to execute all legs results in a rejected trade.

Slippage Aggregation

For a multi-leg position, the total credit or debit is computed as the sum of the executed leg prices.

Let P_i denote the executed price of leg i .

The total entry price becomes:

$$P_{entry} = \sum_{i=1}^k q_i P_i$$

where q_i represents the signed contract quantity.

For credit strategies, the net entry credit is:

$$C_{entry} = -P_{entry}$$

6.4 Exit Fill Model

Exit execution follows similar rules but with additional considerations.

Realistic Close Cost

Closing a position requires crossing the bid–ask spread again.

For credit strategies, the cost to close is typically evaluated using the midpoint price with slippage adjustments.

If the exit price of leg i is $P_{exit,i}$, then the total exit cost becomes:

$$P_{exit} = \sum_{i=1}^k q_i P_{exit,i}$$

The realized profit or loss is then:

$$PnL = C_{entry} - P_{exit}$$

Expired Option Handling

If an option reaches expiration, the simulator determines its intrinsic value.

Let K be the strike price and S_T the underlying price at expiration.

For a call option:

$$IV = \max(S_T - K, 0)$$

For a put option:

$$IV = \max(K - S_T, 0)$$

Expired options are settled according to this intrinsic value.

The final trade PnL reflects the intrinsic settlement of all legs.

6.5 MarketState Generation

The execution engine operates on a per-bar market state representation.

At each timestep t , the simulator constructs a MarketState object containing all information required to evaluate trades.

Conceptually:

```
MarketState
├── timestamp
├── underlying_price
├── feature_vector
├── options_chain
└── portfolio_state
```

Market State Vector

The market feature vector is composed of multi-timeframe inputs:

$$x_t = (x_t^{M1}, x_t^{M5}, x_t^{M15}, x_t^{H1})$$

If neural inference is enabled, model outputs are appended:

$$x_t = (x_t^{M1}, x_t^{M5}, x_t^{M15}, x_t^{H1}, \hat{y}_t)$$

Variables Feeding the Execution Engine

The execution model uses several key inputs:

Variable	Description
underlying price	SPY spot price
options chain	full contract quotes
bid/ask prices	used for fill pricing
portfolio positions	open trades
strategy parameters	template configuration

These variables determine whether trades are feasible and how they are priced.

7. Capital & Risk Constraint System (v4.5 Runtime Rules)

The CondorNet simulation engine enforces a structured capital allocation and risk constraint system designed to approximate institutional portfolio management practices.

The objective of the capital model is to ensure that simulated strategy deployment remains consistent with realistic brokerage margin requirements and portfolio risk controls.

Unlike early versions of the system that allowed only a single open position, the v4.5 architecture supports **multiple concurrent positions across different strategy templates**.

To maintain portfolio stability, all entries must pass a series of **capital and risk gating rules**.

These rules determine:

- how much capital can be deployed,
- how margin requirements are calculated,
- and how strategies compete for portfolio allocation.

7.1 Account Model (Institutional Constraints)

The simulator assumes a baseline trading account with the following parameters.

Starting equity:

$$E_0 = 100,000$$

Brokerage margin leverage is defined as:

$$\lambda = 2$$

Total buying power is therefore:

$$\begin{aligned} BP &= \lambda E_0 \\ BP &= 200,000 \end{aligned}$$

To maintain conservative portfolio exposure, the simulator restricts the fraction of buying power that may be deployed into options strategies.

Let

$$\alpha$$

represent the deployment fraction.

For the current configuration:

$$\alpha = 0.40$$

The maximum deployable capital is therefore:

$$\begin{aligned} BP_{deploy}^{max} &= \alpha BP \\ BP_{deploy}^{max} &= 0.40 \times 200000 \\ BP_{deploy}^{max} &= 80,000 \end{aligned}$$

This constraint ensures that the portfolio never commits more than 40% of its buying power to active strategies.

Definition of Deployed Capital

Deployed capital represents the total margin or collateral currently required by open positions.

Let

$$\mathcal{P}_t$$

denote the set of open positions at time t .

For each position p_i , let M_i denote the required margin.

The deployed capital is therefore:

$$C_{deploy,t} = \sum_{p_i \in \mathcal{P}_t} M_i$$

The portfolio must satisfy the constraint:

$$C_{deploy,t} \leq BP_{deploy}^{max}$$

This ensures that portfolio leverage remains bounded during the simulation.

7.2 Margin Models by Strategy Type

Different options strategies require different margin calculations depending on their risk structure.

The simulator therefore supports several margin models.

Width-Based Margin Model

For defined-risk spreads, the maximum loss is determined by the width of the spread.

Let

$$K_{short}$$

and

$$K_{long}$$

denote the strike prices of the spread legs.

The spread width is:

$$W = |K_{long} - K_{short}|$$

For n contracts, the margin requirement becomes:

$$M = 100 \cdot n \cdot W$$

For credit spreads, the effective margin may be reduced by the credit received:

$$M_{net} = 100 \cdot n \cdot W - C_{entry}$$

This model is used for strategies such as:

- vertical spreads
- iron condors
- butterflies

Percentage-of-Spot Margin Model

For undefined-risk positions such as naked options, margin is approximated as a percentage of the underlying asset price.

Let

$$S_t$$

be the underlying price.

Margin may be approximated as:

$$M = \beta S_t \cdot 100 \cdot n$$

where

$$\beta$$

is a configurable fraction representing broker margin policy.

This model approximates margin requirements for:

- naked short calls
- naked short puts
- certain synthetic positions

Template-Specific Margin Overrides

Some strategies require specialized margin rules.

Templates may therefore override the default margin model via their `CONFIG` definitions.

Example configuration field:

```
margin_model = "width_based"
```

or

```
margin_model = "pct_spot"
```

This mechanism allows the simulator to support a diverse range of strategies while maintaining accurate risk accounting.

7.3 Capital Gating Mathematics

Before opening a new position, the simulator verifies that sufficient capital remains available.

Let

$$C_{deploy,t}$$

denote current deployed capital.

Let

$$M_{new}$$

denote the margin required for the proposed position.

The projected deployed capital after entry becomes:

$$C_{deploy}^{after} = C_{deploy,t} + M_{new}$$

The capital gate condition is therefore:

$$C_{deploy}^{after} \leq BP_{deploy}^{max}$$

If the condition fails, the entry attempt is rejected.

This gating rule ensures that portfolio exposure remains bounded.

7.4 Strategy Pooling and Staged Filters

Because the CondorNet system evaluates many strategy templates simultaneously, a mechanism is required to prioritize which strategies receive capital allocation.

To address this problem, the platform uses a staged filtering process sometimes described as a **digital slider architecture**.

This approach replaces earlier designs that allowed only a single active strategy.

Instead, strategies compete for capital allocation through several evaluation pools.

Pool A — Probability Filter

The first stage filters strategies according to estimated probability of success.

Let

$$P_{win}$$

represent the estimated probability of profit for a candidate strategy.

Strategies must satisfy:

$$P_{win} \in [P_{min}, P_{max}]$$

Example configuration:

$$P_{win} \in [0.75, 1.00]$$

This stage ensures that only strategies with sufficiently high probability of success proceed to later evaluation.

Pool B — Capital Efficiency Ranking

Strategies passing the probability filter are ranked according to capital efficiency metrics.

A common measure is **credit per unit of collateral**:

$$CE = \frac{C_{entry}}{M}$$

Where

- C_{entry} is the credit received
- M is required margin

Additional ranking metrics may include:

- return-to-drawdown ratio
- Sharpe ratio
- expected value

Strategies with higher capital efficiency receive priority for portfolio allocation.

Pool C — Micro-Parameter Optimization

The final stage performs a localized search over nearby parameter configurations.

Let

$$\theta$$

represent the parameter vector of a strategy template.

The optimizer searches a neighborhood:

$$\theta' \in \mathcal{N}(\theta)$$

to identify nearby configurations with improved performance.

This process acts as a **fine-tuning step** before capital is committed.

Institutional Interpretation

The three-stage pooling system effectively implements a **portfolio construction pipeline**:

1. remove low-probability strategies
2. rank remaining strategies by efficiency
3. locally optimize promising configurations

This architecture enables the simulator to allocate capital dynamically across multiple strategy types without relying on rigid single-strategy execution rules.

8. Entry Decision System (Multi-Position, Multi-Template)

The CondorNet platform supports simultaneous evaluation of multiple strategy templates under a shared portfolio constraint system.

At each simulation timestep t , the engine evaluates potential strategy entries using a structured decision pipeline.

The entry decision process determines whether a strategy template may open a new position given:

- market conditions
- strategy configuration
- portfolio constraints
- execution feasibility

This system replaces earlier architectures that allowed only a single open trade.

Instead, the platform evaluates **multiple templates simultaneously** and applies a series of deterministic decision gates.

Formally, an entry action can be expressed as:

$$a_t = ENTER(s, \theta)$$

where

Symbol	Meaning
s	strategy template
θ	parameter configuration

If all gating conditions pass, a new position p_{new} is created and added to the portfolio:

$$\mathcal{P}_{t+1} = \mathcal{P}_t \cup \{p_{new}\}$$

8.1 Entry Decision Primitives

Entry decisions are evaluated using three conceptual components:

1. **Signals**
 2. **Constraints**
 3. **Feasibility checks**
-

Signals

Signals determine whether the strategy desires to enter a position under current market conditions.

Examples include:

- model predictions
- volatility regime indicators
- technical signals
- probability-of-profit estimates

Signal evaluation produces a boolean decision:

$$Signal(s, s_t) \in \{0,1\}$$

If the signal evaluates to false, the strategy does not attempt entry.

Constraints

Constraints represent policy rules imposed by the simulator.

Examples include:

- maximum number of open positions
- cooldown periods
- strategy class restrictions

These constraints enforce portfolio discipline.

Feasibility Checks

Feasibility checks verify that the strategy can be executed in the current market environment.

Examples include:

- required option contracts exist
- sufficient liquidity
- capital availability
- atomic multi-leg fill possibility

Only when all three categories succeed does the simulator proceed with trade creation.

8.2 Gate Taxonomy

The entry pipeline consists of a sequence of decision gates.

Each gate represents a specific validation stage.

If a gate fails, the entry attempt is rejected and a failure counter is incremented.

Policy Finite Gate

Ensures that the strategy configuration contains finite numeric values.

Invalid parameters such as NaN or infinite values trigger rejection.

$$\text{finite}(\theta) = \text{True}$$

Cooldown Gate

Some strategies require a cooldown period between entries.

Let

$$t_{last}$$

be the time of the most recent position opened by the strategy.

Entry is permitted only if:

$$t - t_{last} \geq \text{cooldown}$$

Entry Signal Gate

This gate evaluates the primary strategy signal.

$$\text{Signal}(s, s_t) = \text{True}$$

If the signal condition fails, no entry is attempted.

Probability-of-Profit Gate

Strategies may require a minimum probability of profit.

Let

$$P_{win}$$

denote the estimated probability of success.

Entry is permitted if:

$$P_{win} \geq P_{min}$$

Typical thresholds range between 0.60 and 0.80 depending on strategy type.

Strategy Abstain Gate

Some strategies may abstain under certain market regimes.

For example:

- extreme volatility
- illiquid option chains
- incompatible model predictions

The abstain rule acts as an additional signal filter.

Chain Integrity Gate

Before constructing a position, the simulator verifies that the option chain contains valid data.

Checks include:

- bid and ask prices present
- valid strike ordering
- non-zero spreads
- positive liquidity

If the chain fails validation, the entry attempt is rejected.

Legs Found Gate

Each strategy template defines a set of required option legs.

The simulator must locate contracts matching these specifications.

If the required contracts cannot be located, the entry fails.

$$|L_{found}| = |L_{required}|$$

Atomic Fill Gate

Multi-leg strategies must be filled atomically.

If any leg fails execution feasibility, the entire trade is rejected.

$$Fill(L) = True$$

Credit Floor Gate

Credit strategies must produce a minimum premium.

Let

$$C_{entry}$$

be the net credit of the position.

The condition becomes:

$$C_{entry} \geq C_{min}$$

This prevents trades with negligible reward.

Capital Gate

The portfolio must have sufficient margin capacity.

Let

$$C_{deploy,t}$$

be current deployed capital and M_{new} the required margin.

Entry is permitted if:

$$C_{deploy,t} + M_{new} \leq BP_{deploy}^{max}$$

Sizing Consistency Gate

The system verifies that the number of contracts requested by the strategy is consistent with margin requirements and template configuration.

If inconsistencies are detected, the simulator may halt execution to prevent data corruption.

8.3 Parameter Decoding and Clamping Rules

Strategy parameters may originate from:

- template configuration
- optimizer adjustments
- command-line overrides

These parameters must be validated and clamped to acceptable ranges.

Offset Parameters

Strike offsets must remain within realistic bounds.

$$offset_{min} \leq offset \leq offset_{max}$$

Offsets outside the allowable range are clamped.

Spread Width Parameters

Spread widths must satisfy:

$$width > 0$$

and remain within available strike distances in the option chain.

DTE Bounds

Strategies specify target days-to-expiration.

Let

$$DTE_{target}$$

be the requested expiration distance.

The selected contract must satisfy:

$$DTE_{min} \leq DTE \leq DTE_{max}$$

Template Parameter Overrides

The optimizer may override template parameters. These overrides are applied after defaults are merged. Conceptually:

$$\theta = merge(CONFIG_{defaults}, CONFIG_{template}, \theta_{optimizer})$$

8.4 Leg Discovery and Selection

Once the strategy passes the decision gates, the simulator must locate the specific option contracts required for the position.

Template Scoring

When multiple candidate contracts exist, the simulator ranks them according to a scoring function.

Typical scoring criteria include:

- delta proximity
- strike distance
- liquidity
- bid–ask spread

Delta and Strike Targeting

Strategies frequently target specific option deltas.

Let

$$\Delta_{target}$$

represent the desired delta.

The selected contract minimizes:

$$|\Delta - \Delta_{target}|$$

Alternatively, rank-based targeting may be used.

Example ranks:

Rank	Meaning
0	ATM strike
+1	first OTM
+2	second OTM

Liquidity Validation

Before constructing the trade, candidate contracts must satisfy liquidity requirements.

Typical criteria include:

- minimum open interest
- maximum spread percentage
- positive bid price

Contracts failing these conditions are rejected.

8.5 OPEN Event Emission Schema

When a strategy successfully opens a position, the simulator emits an OPEN event.

This event records the complete state of the trade at the time of entry.

Events are written to the simulation log.

Event Row Contract

Each OPEN event is represented as a structured record.

Example schema:

Field	Description
event_type	OPEN
timestamp	entry time
trade_id	unique identifier
strategy	template name
contracts	number of contracts
entry_credit	premium received
margin	required collateral
legs	option contract identifiers

Required Keys

All OPEN events must include the following keys:

- event_type
- timestamp
- trade_id
- strategy
- entry_credit
- margin

These fields ensure that each trade can be uniquely identified and analyzed.

Linking to CLOSE Events

Every OPEN event generates a unique trade identifier.

This identifier is used to associate future lifecycle events with the original position.

Example linkage:

```
OPEN  trade_id = 128  
CLOSE trade_id = 128
```

This structure allows the simulator to reconstruct the full lifecycle of each trade.

9. Exit / Hold / Modify / Add Lifecycle (Policy-Aware)

After a position has been opened, the CondorNet simulation engine must determine how the trade evolves through time.

At each timestep t , the simulator evaluates all open positions and determines whether they should:

- remain open (**HOLD**),
- be closed (**EXIT**),
- be adjusted (**MODIFY**),
- or be expanded (**ADD**).

These actions form the **post-entry lifecycle decision system**. Formally, for each open position p_i , the action space is:

$$a_t^{(i)} \in \{\text{HOLD}, \text{EXIT}, \text{MODIFY}, \text{ADD}\}$$

where

Action	Meaning
HOLD	maintain the current position
EXIT	close the entire position
MODIFY	adjust structure (roll, hedge, change strikes)
ADD	increase position size or add legs

The lifecycle system may operate through deterministic policy rules or through model-assisted decision mechanisms.

9.1 Exit System Overview

The exit system determines when positions should be closed.

Exit decisions may arise from two sources:

1. **Deterministic policy rules**
2. **Model-assisted signals**

Deterministic Exit Rules

Most strategies define explicit exit criteria.

Examples include:

- profit target reached
- stop-loss triggered
- expiration proximity
- liquidity deterioration

These rules are evaluated deterministically.

For a position p_i , an exit occurs when:

$$Exit(p_i, s_t) = True$$

Model-Assisted Exit Signals

In advanced configurations, exit decisions may incorporate predictions from machine learning models.

For example, a neural model may estimate future volatility conditions.

If the predicted regime becomes unfavorable, the system may recommend closing the position early.

Such signals augment the deterministic policy rules but do not replace them.

9.2 Stop-Loss Mathematics

Stop-loss rules limit the maximum loss that a strategy may incur.

Two primary stop-loss definitions are supported.

Dollar Stop

A fixed dollar threshold relative to the entry credit.

Let

$$C_{entry}$$

be the premium received at entry.

Let

$$PnL_t$$

be the current unrealized profit or loss. The stop-loss condition is triggered when:

$$PnL_t \leq -L_{max}$$

where

$$L_{max}$$

is the maximum permitted loss.

Multiplier Stop

Some strategies define stop-loss thresholds as a multiple of the entry credit.

Let

$$m$$

be the stop multiplier.

The stop-loss condition becomes:

$$PnL_t \leq -m \cdot C_{entry}$$

Typical values include:

Multiplier	Meaning
1.5	moderate stop
2.0	aggressive stop
3.0	loose stop

Per-Template Stop Policies

Each strategy template may define its own stop-loss configuration.

Example parameters include:

- stop multiplier
- absolute dollar limit
- time-based stop overrides

These values are defined in the template `CONFIG` structure.

9.3 Profit Target Mathematics

Profit targets define the conditions under which positions are closed early to capture gains.

Two main target types are supported.

Fixed Dollar Target

The position is closed when profit reaches a predefined dollar threshold.

$$PnL_t \geq T$$

where

$$T$$

is the profit target.

Percentage Credit Capture

More commonly, credit strategies close after capturing a percentage of the initial premium.

Let

$$\gamma$$

be the capture fraction.

The target becomes:

$$PnL_t \geq \gamma \cdot C_{entry}$$

Typical values include:

Capture Fraction	Interpretation
0.25	conservative exit
0.50	standard target
0.75	aggressive hold

This approach reduces time exposure while locking in profits.

9.4 Holding Zones and Time-Based Exits

In addition to price-based exits, strategies may incorporate time-based constraints.

These rules prevent positions from remaining open under unfavorable temporal conditions.

DTE Boundary Conditions

Each position has a remaining number of days to expiration:

$$DTE_t$$

Strategies may enforce minimum holding windows or maximum holding durations.

Example rules:

$$DTE_t \leq DTE_{exit}$$

which forces exit when expiration approaches.

Expiry Risk Handling

Holding positions near expiration introduces several risks:

- gamma acceleration
- liquidity deterioration
- assignment risk

To mitigate these risks, strategies may close positions before expiration.

Typical exit thresholds include:

Strategy Type	Exit DTE
short premium	7–14 days
spreads	5–10 days
long options	near expiration

Liquidity Risk Handling

Positions may also be closed early if liquidity deteriorates.

Indicators include:

- widening bid–ask spreads
- falling open interest
- reduced market depth

These conditions increase execution risk and therefore justify early exit.

9.5 Modify / Add Actions

Beyond simple exits, the simulator supports structural adjustments to positions.

These adjustments are represented as discrete actions within the lifecycle system.

Modify Actions

Modify actions alter the structure of an existing position.

Examples include:

- rolling strikes
- extending expiration
- adjusting spread width
- adding protective wings

Formally:

$$p_i \rightarrow p'_i$$

where the modified position replaces the original structure.

Add Actions

Add actions increase the exposure of an existing position.

Examples include:

- increasing contract size
- adding additional spreads
- layering positions

This action expands the leg set:

$$L'_i = L_i \cup L_{new}$$

Position Mutation

Both modify and add actions represent **position mutations**.

A mutation changes the structure or size of a position without closing it entirely.

In the context of dataset generation, mutations must be recorded explicitly so that models can learn adjustment behavior.

9.6 CLOSE Event Standardization Contract

When a position is fully exited, the simulator emits a CLOSE event.

This event records the final outcome of the trade.

Required Fields

Each CLOSE event must contain the following fields:

Field	Description
action	CLOSE
trade_id	unique identifier
timestamp	exit time
heldBars	duration of the trade
realized_pnl	final profit or loss
exit_reason	rule that triggered the exit

Example event:

```
{  
  action: "CLOSE",  
  trade_id: 128,  
  heldBars: 47,  
  realized_pnl: 315.50,  
  reason: "profit_target"  
}
```

Lifecycle Integrity

Each CLOSE event must correspond to a prior OPEN event.

Formally:

$$trade_id_{close} \in trade_id_{open}$$

This constraint ensures that trade lifecycles remain well-defined.

Dataset Export Requirements

Clean state-action datasets require standardized lifecycle events.

For each trade, the simulator must record:

1. OPEN event
2. intermediate HOLD states
3. final CLOSE event

These records enable the reconstruction of complete trajectories:

$$(s_t, a_t, r_t, s_{t+1})$$

which are used for reinforcement learning or supervised policy training.

10. Optimizer System (Per-Strategy Parameter Sweep)

The CondorNet platform includes a strategy optimization system designed to explore the performance characteristics of strategy templates across a parameter space.

The optimizer is implemented in:

```
strategy_optimizer.py
```

Its primary purpose is to generate **performance surfaces** for strategy configurations rather than to produce a single “optimal” parameter set.

Each strategy template s defines a parameter vector:

$$\theta = (\theta_1, \theta_2, \dots, \theta_k)$$

where each parameter corresponds to a configurable element of the strategy such as stop-loss thresholds, profit targets, or position sizing.

The optimizer evaluates the strategy across the parameter grid:

$$\theta \in \Theta_s$$

where Θ_s represents the discrete parameter search space associated with template s .

For each configuration, the simulator produces performance metrics which are then used to construct a performance surface.

10.1 Purpose: Optimization Surface Generation

The optimizer is designed to **map the performance landscape of strategy configurations** rather than to perform classical overfitting.

Traditional backtesting systems often attempt to identify a single “best” configuration based on historical data.

This approach frequently leads to overfitting.

Instead, the CondorNet optimizer generates a **surface of results** across parameter combinations.

Formally, for strategy s , the optimizer estimates the function:

$$J_s(\theta)$$

where

$$J_s(\theta)$$

represents a performance metric computed by the simulator.

The result is a mapping:

$$\Theta_s \rightarrow \mathbb{R}$$

which can be visualized as a performance surface.

Researchers then examine the stability of this surface rather than relying on a single parameter configuration.

10.2 strategy_optimizer.py Architecture

The optimizer is implemented as an external controller around the core simulation engine.

Conceptually, it operates as:

```
optimizer
  ↓
run_backtest()
  ↓
metrics collection
  ↓
results ranking
```

The optimizer integrates with the simulation engine through repeated calls to the backtesting routine defined in:

```
condor_brain_backtest_v45.py
```

Inputs

Typical optimizer inputs include:

Input	Description
strategy template	name of strategy module
dataset timeframe	simulation timeframe
parameter grid	parameter search space
dataset paths	market data locations
runtime options	inference flags and execution settings

Outputs

Each optimizer run produces:

- parameter configuration
- performance metrics
- constraint statistics
- ranking score

These results are stored for later analysis.

Interactive Mode

The current optimizer is designed primarily for **interactive research use**.

Researchers may manually explore the parameter grid, inspect results, and adjust search ranges dynamically.

Batch Mode (Future)

Future versions may include a fully automated batch mode capable of:

- scanning all strategy templates
- exploring parameter grids in parallel
- generating large-scale optimization reports

This would enable overnight optimization runs across many strategies.

10.3 Parameter Grid Definition

The parameter grid defines the search space explored by the optimizer.

Each strategy template exposes a set of tunable parameters.

Current parameters include:

Parameter	Meaning
stop_loss_dollar	absolute stop-loss threshold
profit_target	target profit level
max_positions	maximum concurrent trades
cooldownBars	minimum delay between entries
stop_loss_mult	stop-loss multiplier
max_contracts	position size cap

Each parameter may take multiple values.

For example:

```
profit_target ∈ {0.25, 0.50, 0.75}  
stop_loss_mult ∈ {1.5, 2.0, 3.0}
```

The optimizer evaluates all combinations of the parameter grid.

Formally:

$$\Theta_s = \theta_1 \times \theta_2 \times \cdots \times \theta_k$$

where each dimension corresponds to a parameter domain.

Parameter Mapping

Parameters may be applied through either:

1. **Template configuration overrides**
2. **Command-line arguments**

Template parameters modify the strategy `CONFIG` dictionary.

Command-line parameters modify runtime behavior of the simulation engine.

The final configuration used for a run is:

$$\theta = \text{merge}(\text{CONFIG}_{\text{template}}, \theta_{\text{optimizer}})$$

10.4 Optimization Loop Mechanics

The optimizer performs repeated backtest runs under different parameter configurations.

Conceptually:

```
for  $\theta$  in parameter_grid:
    run_backtest( $\theta$ )
    collect_metrics()
    store_result()
```

Run Cloning and State Isolation

Each run must operate in isolation to prevent contamination of results.

Therefore the optimizer resets:

- portfolio state
- dataset iterators
- execution counters
- event logs

before each run.

Formally, each run is treated as an independent experiment:

$$R_i = \text{Backtest}(s, \theta_i)$$

Metrics Collection

For each run R_i , the simulator produces performance metrics including:

- total return
- maximum drawdown
- win rate
- number of trades
- average holding time

These metrics form the evaluation vector:

$$m_i = (\text{return}_i, \text{drawdown}_i, \text{winrate}_i, \dots)$$

Ranking Function

To compare parameter configurations, the optimizer computes a ranking score.

A common metric is the **net-profit to drawdown ratio**:

$$\text{Score} = \frac{NP}{DD}$$

where

Symbol	Meaning
NP	net profit
DD	maximum drawdown

Higher scores indicate more favorable risk-adjusted performance.

Alternative ranking functions may also be used depending on research objectives.

10.5 Optimization Output Schema

The optimizer stores results for each parameter configuration in a structured table.

Each row corresponds to one simulation run.

Typical columns include:

Column	Description
strategy	template name
parameters	parameter vector
net_profit	total profit
max_drawdown	worst drawdown
win_rate	percentage of profitable trades
avg_hold	average holding duration
trades	number of trades executed
score	ranking metric

Additional statistics may include:

- gate failure counts
- capital constraint hits
- average credit received

Storage Layout

Optimization results are typically written to the following directory structure:

```
reports/  
  optimization/  
    <strategy_name>/  
      <timeframe>.csv
```

Example:

```
reports/optimization/iron_condor/m5.csv
```

Each file represents the optimization surface for a given strategy and timeframe.

10.6 Applying Best Configuration Back to Template

Once promising configurations have been identified, researchers may choose to apply these parameters directly to the strategy template.

This process modifies the template's `CONFIG` dictionary.

Safe Template Patching

Configuration updates must be performed carefully to avoid corrupting strategy definitions.

Typical procedure:

1. load existing template
2. update relevant parameters
3. write modified configuration back to file

Example parameters that may be patched:

- stop loss multiplier
 - profit target
 - contract sizing limits
-

Traceability and Versioning

To maintain research integrity, configuration updates should be traceable.

Recommended practices include:

- recording optimization run identifiers
- storing commit hashes
- documenting parameter changes

For example:

```
CONFIG["profit_target"] = 0.50  
CONFIG["stop_loss_mult"] = 2.0
```

Accompanied by metadata such as:

```
# updated via optimizer run 2026-02-27  
# commit hash: a927128
```

This ensures that strategy evolution remains auditable.

11. Metrics, Surfaces, and Institutional Scoreboards

The CondorNet simulator produces a large number of quantitative outputs describing strategy performance, execution quality, and dataset integrity.

These outputs serve two primary purposes:

1. **Strategy evaluation** — determining which parameter configurations perform well.
2. **Research validation** — ensuring that the simulation environment behaves realistically.

Metrics are therefore divided into two categories:

- **performance metrics** — evaluating trading results
- **truth metrics** — evaluating the integrity of the simulation dataset

The platform also supports the analysis of **optimization surfaces**, which provide insight into the stability of strategy parameters.

11.1 Core Performance Metrics

Core performance metrics describe how a strategy or portfolio performs over the simulation horizon.

Let

$$V_t$$

denote portfolio equity at time t , and let

$$V_0$$

represent initial capital.

Total Return

Total return measures the overall change in portfolio value over the simulation period.

$$R_{total} = \frac{V_T - V_0}{V_0}$$

where

Symbol	Meaning
V_T	final portfolio value
V_0	initial portfolio value

Total return provides a basic measure of profitability.

Maximum Drawdown

Maximum drawdown measures the largest peak-to-trough decline in portfolio value.

Let

$$V_{peak}(t) = \max_{u \leq t} V_u$$

The drawdown at time t is:

$$DD_t = \frac{V_{peak}(t) - V_t}{V_{peak}(t)}$$

Maximum drawdown is defined as:

$$DD_{max} = \max_t DD_t$$

This metric captures the worst loss experienced during the simulation.

Sharpe Ratio (If Implemented)

The Sharpe ratio measures risk-adjusted returns.

Let

$$r_t$$

represent returns per period.

The Sharpe ratio is defined as:

$$Sharpe = \frac{\mathbb{E}[r_t - r_f]}{\sigma(r_t)}$$

where

Symbol	Meaning
r_f	risk-free rate
$\sigma(r_t)$	standard deviation of returns

This metric indicates how efficiently the strategy converts risk into return.

Sortino Ratio (Optional)

The Sortino ratio penalizes only downside volatility.

$$Sortino = \frac{\mathbb{E}[r_t - r_f]}{\sigma_{down}(r_t)}$$

where

$$\sigma_{down}$$

is the standard deviation of negative returns.

This metric is particularly relevant for asymmetric payoff strategies such as short volatility trades.

Win Rate

Win rate measures the fraction of trades that produce positive profit.

Let

$$N_{win}$$

be the number of profitable trades and

$$N_{total}$$

the total number of trades.

The win rate is:

$$WR = \frac{N_{win}}{N_{total}}$$

Win rate alone is not sufficient to evaluate performance, but it provides useful context.

Average Holding Time

Average holding time measures how long trades remain open.

Let

$$h_i$$

represent the number of bars held by trade i .

The average holding duration is:

$$H_{avg} = \frac{1}{N_{total}} \sum_{i=1}^{N_{total}} h_i$$

This metric helps characterize the time exposure of strategies.

Capital Efficiency

Capital efficiency measures the return generated per unit of deployed margin.

Let

$$NP$$

represent net profit and

$$M_{avg}$$

represent average deployed margin.

Capital efficiency is defined as:

$$CE = \frac{NP}{M_{avg}}$$

This metric is particularly important when comparing strategies with different margin requirements.

Tail Risk Proxies

Tail risk metrics capture the severity of extreme losses.

Examples include:

- worst single trade loss
- largest daily portfolio loss
- loss quantiles

For example, the 95% loss quantile is:

$$Q_{0.95} = \text{quantile}_{0.95}(\text{losses})$$

These metrics help assess exposure to rare but severe outcomes.

11.2 “Truth Metrics” (Dataset Quality Metrics)

In addition to performance metrics, the CondorNet platform tracks metrics related to the integrity of the simulation environment.

These metrics help ensure that the dataset generated by the simulator reflects realistic market conditions.

Gate Pass Rates

Each entry attempt passes through multiple decision gates.

The simulator tracks how often each gate succeeds or fails.

For gate g , the pass rate is:

$$\text{PassRate}_g = \frac{N_{\text{pass},g}}{N_{\text{attempt},g}}$$

Examples of gates include:

- entry signal
- probability-of-profit
- chain integrity
- capital constraint

These metrics help diagnose overly restrictive or overly permissive strategy rules.

Average Slippage by Structure Class

Execution quality is monitored through slippage metrics.

Let

$$P_{expected}$$

represent the midpoint price and

$$P_{fill}$$

the actual simulated fill price.

Slippage is defined as:

$$Slip = P_{fill} - P_{expected}$$

Average slippage may be computed by strategy class.

$$Slip_{avg}(c) = \frac{1}{N_c} \sum_{i=1}^{N_c} Slip_i$$

where c represents the structure class.

This metric reveals which strategy types suffer the most execution friction.

Atomic Fill Success Rate

Because multi-leg strategies require simultaneous execution, the simulator tracks how often atomic fills succeed.

Let

$$N_{atomic}$$

represent successful atomic fills and

$$N_{attempt}$$

the number of attempts.

The atomic fill success rate is:

$$AFSR = \frac{N_{atomic}}{N_{attempt}}$$

Low values may indicate unrealistic strike targeting or liquidity constraints.

Chain Integrity Failure Rate

Option chain data may occasionally contain missing or invalid entries.

The simulator tracks the fraction of entry attempts rejected due to chain integrity failures.

$$CIFR = \frac{N_{chain_fail}}{N_{attempt}}$$

Monitoring this metric helps detect data quality issues.

11.3 Optimization Surfaces

An optimization surface represents the relationship between strategy performance and parameter values.

For a strategy s with parameter vector θ , the optimizer estimates:

$$J_s(\theta)$$

where J_s represents a performance metric such as risk-adjusted return.

If two parameters are varied simultaneously, the result can be visualized as a surface:

$$J(\theta_1, \theta_2)$$

Such surfaces provide insight into how sensitive strategy performance is to parameter changes.

Surface Interpretation

Stable strategies typically exhibit **broad plateaus** in their optimization surfaces.

This indicates that performance remains strong across a range of parameter values.

In contrast, narrow peaks may indicate overfitting.

Detecting Overfitting Within a Single Year

Because the simulator may initially operate on a single year of data, it is important to detect overfitting.

Indicators of overfitting include:

- extreme sensitivity to small parameter changes
- sharp isolated peaks in performance surfaces
- inconsistent performance across nearby parameter values

Researchers should therefore prioritize configurations that lie within **stable regions of the surface** rather than selecting the single best-performing point.

12. State–Action Trajectory Dataset Factory (Primary Product)

While traditional backtesting systems primarily produce trade summaries or performance reports, the CondorNet platform is designed to generate structured datasets suitable for machine learning training.

The simulation engine therefore functions not only as a strategy evaluation tool but also as a **trajectory dataset generator**.

The output dataset captures the evolution of the trading environment through time and records the actions taken by the simulator.

Formally, the dataset consists of a sequence of tuples:

$$(s_t, a_t, r_t, s_{t+1})$$

Where

Symbol	Meaning
s_t	state at time t
a_t	action taken
r_t	reward observed
s_{t+1}	next state

This structure allows the dataset to support both:

- reinforcement learning
- supervised policy learning

The trajectory dataset is therefore considered the **primary research output** of the CondorNet platform.

12.1 Why `trades.csv` Is Insufficient

A traditional backtesting output typically consists of a file such as:

`trades.csv`

which records summary information for each completed trade.

Example fields include:

- entry timestamp
- exit timestamp
- profit or loss
- strategy name

While useful for performance reporting, such summaries omit the sequential decision context that produced the trade.

Specifically, a trade summary does not capture:

- intermediate holding states
- rejected entry attempts
- portfolio conditions at decision time
- market features available to the strategy

As a result, trade-level summaries are insufficient for training machine learning models.

To address this limitation, the CondorNet simulator records **state–action trajectories** describing the full decision process at every timestep.

12.2 State Vector Composition

The state vector s_t represents the information available to the simulator at time t .

The state vector is composed of three primary components:

1. **market state**
 2. **portfolio state**
 3. **strategy context**
-

Market State

The market state contains multi-timeframe features derived from the underlying asset and options market.

Let

$$x_t$$

represent the market feature vector.

Example features include:

- price returns
- volatility indicators
- pivot features
- neural model predictions

Formally:

$$x_t = (x_t^{M1}, x_t^{M5}, x_t^{M15}, x_t^{H1})$$

Portfolio State

The portfolio state summarizes currently active positions.

Let

$$\mathcal{P}_t$$

denote the set of open positions.

A summary function

$$\phi(\mathcal{P}_t)$$

maps the portfolio into a fixed-length feature vector.

Typical portfolio features include:

- number of open positions
 - deployed capital
 - portfolio delta
 - portfolio gamma
 - unrealized PnL
-

Strategy Context

Additional information describing the strategy under evaluation may also be included.

Examples include:

- strategy template identifier
 - parameter configuration
 - cooldown state
-

Combined State Vector

The full state representation is therefore:

$$s_t = (x_t, \phi(\mathcal{P}_t), c_t)$$

where

$$c_t$$

represents contextual features.

12.3 Action Space Definition

At each timestep the simulator selects an action from a discrete action set.

The action space is defined as:

$$A = \{\text{ENTER}, \text{HOLD}, \text{EXIT}, \text{MODIFY}, \text{ADD}\}$$

ENTER

Opens a new position using a specific strategy template.

$$a_t = \text{ENTER}(s)$$

where

$$s$$

is the selected strategy template.

HOLD

Maintains an existing position without modification.

$$a_t = \text{HOLD}$$

This action represents the decision to continue holding a trade.

EXIT

Closes an existing position.

$$a_t = \text{EXIT}(\text{reason})$$

where the reason may include:

- profit target
- stop loss

- expiration rule
 - risk management decision
-

MODIFY

Alters the structure of an existing position.

Examples include:

- rolling expiration
 - adjusting strike levels
 - adding protective legs
-

ADD

Expands an existing position.

Examples include:

- increasing contract size
- layering additional spreads

These actions enable the simulator to represent more complex portfolio management behavior.

12.4 Reward Definition

The reward function defines the objective used by learning algorithms.

Rewards are derived from changes in portfolio value.

Let

$$V_t$$

denote portfolio equity at time t .

Immediate Reward

The simplest reward formulation measures the change in equity between timesteps.

$$r_t = V_{t+1} - V_t$$

This captures incremental portfolio gains or losses.

Terminal Reward

Some learning frameworks emphasize final trade outcomes.

Let

$$PnL_i$$

represent the final profit of trade i . The terminal reward is:

$$r_{terminal} = PnL_i$$

This reward occurs when the position is closed.

Risk-Adjusted Reward

To encourage stable strategies, rewards may incorporate risk penalties.

Example:

$$r_t = (V_{t+1} - V_t) - \lambda \cdot Risk_t$$

where

$$Risk_t$$

may represent:

- drawdown
 - capital usage
 - volatility exposure
-

Penalty Shaping

Penalty shaping discourages undesirable behaviors.

Examples include penalties for:

- excessive leverage
- constraint violations
- excessive trade frequency

Formally:

$$r_t = \Delta V_t - P_t$$

where P_t represents penalties applied at time t .

12.5 Dataset Schemas

The simulator generates structured datasets describing both state trajectories and discrete events.

Two primary dataset types are produced.

StateActionRecord Dataset

This dataset contains sequential decision data.

Typical columns include:

Column	Description
timestamp	simulation time
state_vector	encoded state features
action	action taken
reward	reward value
next_state	next state representation
trade_id	associated position

The dataset may be stored in CSV or Parquet format.

EventLog Dataset

The event log records lifecycle events associated with trades.

Typical event types include:

- OPEN
- CLOSE
- MODIFY
- ADD

Example schema:

Column	Description
timestamp	event time
event_type	lifecycle event
trade_id	trade identifier
strategy	strategy template
parameters	parameter configuration
realized_pnl	profit or loss

These logs allow reconstruction of trade lifecycles.

12.6 Joining Logic

Because the simulator may hold multiple positions simultaneously, dataset records must be linked carefully.

Linking Records via Trade ID

Each position receives a unique identifier:

trade_id

All lifecycle events and state records reference this identifier.

This allows the full history of each trade to be reconstructed.

Multiple Concurrent Positions

At time t , the portfolio may contain multiple positions:

$$\mathcal{P}_t = \{p_1, p_2, \dots, p_n\}$$

The dataset must therefore represent multiple simultaneous trajectories.

Each record corresponds to a state–action pair associated with a particular trade.

The full dataset therefore contains interleaved trajectories representing different positions evolving over time.

13. Auditing and Explainability (Predicate / Sets / Supersets)

Institutional trading research systems must support full auditability of decision logic.

Unlike traditional backtesters that only produce trade summaries, the CondorNet platform records detailed explanations for every decision made during simulation.

These explanations allow researchers to answer questions such as:

- Why did the system open this trade?
- Why were other strategies rejected?
- Which constraints prevented an entry?
- Which rule triggered an exit?

To support this capability, the simulator produces structured audit artifacts and enforces strict validation rules during execution.

The audit system is based on a hierarchical evaluation model consisting of:

- **predicates**
- **sets**
- **supersets**

Each entry decision is decomposed into these components to explain the reasoning process behind the simulator’s actions.

13.1 Required Audit Artifacts

Several audit artifacts are generated during simulation.

These artifacts capture the full reasoning process used by the decision engine.

Per-Bar Decision Trace

For every simulation bar, the simulator records a structured decision trace.

These traces are typically written to a JSON Lines file:

```
reports/bar_trace.jsonl
```

Each record describes the evaluation state for that timestep.

Example fields include:

Field	Description
timestamp	simulation time
candidate_strategies	strategies evaluated
gates_passed	gates successfully passed
gates_failed	gates rejected
selected_action	action executed

Example trace entry:

```
{
  "timestamp": "2025-06-12T10:35:00",
  "candidates": ["iron_condor", "short_strangle"],
  "gates_passed": ["entry_signal", "chain_integrity"],
  "gates_failed": ["capital_gate"],
  "selected_action": "HOLD"
}
```

This record allows the full decision process for each timestep to be reconstructed.

Per-Entry Decomposition

When an entry attempt occurs, the simulator records a detailed explanation of the decision.

The evaluation is broken into three logical layers:

Predicates

Predicates represent individual boolean conditions.

Examples include:

- entry signal satisfied
- option chain valid
- sufficient liquidity
- margin available

Each predicate evaluates to either **true** or **false**.

Sets

Sets group related predicates together.

For example:

$$Set_{entry} = \{signal_valid, chain_valid, capital_available\}$$

A set is considered valid only if all predicates pass.

Supersets

Supersets represent the full decision conditions required for entry.

$$Superset = \bigcap_i Set_i$$

In other words, the superset is satisfied only when all required sets pass.

This hierarchical structure allows the simulator to explain precisely why a trade was or was not executed.

Per-Trade Lifecycle Trace

In addition to entry traces, the simulator records the full lifecycle of each trade.

Lifecycle traces include:

- entry conditions
- holding state evolution
- exit trigger
- final profit or loss

These traces allow researchers to inspect the behavior of individual trades across time.

Example lifecycle log:

Field	Description
trade_id	unique identifier
entry_timestamp	trade start
exit_timestamp	trade close
exit_reason	rule that triggered exit
realized_pnl	final profit

13.2 Debug Halting Rules (Stop-the-World Triggers)

To maintain simulation correctness, the system enforces strict validation rules during execution.

If certain invariant violations occur, the simulator halts immediately.

These conditions are known as **stop-the-world triggers**.

Sizing Inconsistency

Position sizing must remain internally consistent.

For example:

$$contracts_{short} = contracts_{long}$$

for vertical spreads.

If a mismatch occurs between expected and actual contract counts, the simulator halts execution.

This prevents corrupted trade structures.

NaN Policy Outputs

Policy models must produce finite numeric outputs.

If any model output contains a NaN value:

$$output = NaN$$

the simulator stops execution.

This prevents undefined behavior from propagating through the system.

Negative Collateral

Margin usage must remain non-negative.

Let

$$M_t$$

represent total deployed margin.

The invariant is:

$$M_t \geq 0$$

If the simulator detects negative collateral values, execution is halted.

Such conditions indicate a logic error in margin calculations.

Broken Chain Invariants

Option chains must satisfy structural constraints.

Examples include:

- valid strike ordering
- non-negative option prices
- consistent expiration dates

If these invariants are violated, the simulator halts to prevent invalid trades.

13.3 Reproducibility Requirements

For audit results to be considered valid, simulation runs must be reproducible.

This requires strict control over randomness and configuration.

Random Seeds

All stochastic components must be initialized with deterministic seeds.

Let

seed

represent the random seed used for the run.

Using fixed seeds ensures that identical simulations produce identical results.

Deterministic Selection Order

When multiple strategies or candidate trades are available, selection order must be deterministic.

For example, candidates may be sorted by:

- strategy name
- score
- timestamp

This prevents non-deterministic behavior caused by unordered iteration.

Configuration Snapshots

Each simulation run must record the exact configuration used.

Typical snapshot elements include:

- strategy parameters
- optimizer settings

- dataset paths
- runtime flags

These snapshots ensure that historical runs can be reproduced precisely.

14. Command Line Interface Reference (Backtester)

The CondorNet / CondorBrain backtesting engine is launched through the primary runtime entrypoint:

```
condor_brain_backtest_v45.py
```

The runtime interface is defined in `main()` via Python `argparse`. The CLI governs:

- input data selection
- model and ruleset selection
- hybrid logic toggles
- strategy filtering
- execution reality controls
- v4.3 multi-timeframe inference mode
- day-of-week DTE caps
- runtime limits and verbosity

The command-line interface therefore acts as the operational control surface for the simulator.

14.1 Full CLI Argument List (by Functional Group)

Data Paths and Core Inputs

These arguments determine what market data and supporting files are loaded.

Argument	Type	Default	Meaning
<code>--input</code>	str	None	Path to input CSV data
<code>--data</code>	str	None	Alias for <code>--input</code>
<code>--model</code>	str	None	Path to model checkpoint (.pth)
<code>--ruleset</code>	str	None	Path to ruleset YAML
<code>--rules-only</code>	flag	False	Run in logic-only mode without neural model

The backtester accepts either `--input` or `--data` as the primary single-file dataset argument in the legacy path.

Hybrid Logic and Core Runtime Toggles

These arguments enable or disable major functional blocks in the decision engine.

Argument	Type	Default	Meaning
--use-fuzzy-sizing	flag	False	Enable 11-factor fuzzy position sizing
--use-trade-rules	flag	False	Enable rule-based entry/exit logic
--use-diffusion	flag	False	Enable diffusion-based parameter refinement
--limit	int	None	Limit the number of bars to simulate
--max-positions	int	5	Maximum simultaneous open positions
--verbose	flag	False	Print detailed per-bar output

These switches control whether the runtime behaves as:

- neural only,
- rules only,
- or a hybrid policy stack.

Strategy Selection Controls

These arguments restrict or filter which strategy classes may trade.

Argument	Type	Default	Meaning
--strategies	str	"all"	Allowed strategy set
--strategyomit	list[str]	None	Excluded strategy classes
--profittargets	flag	False	Enable per-strategy dollar profit targets
--optimize	flag	False	Interactive parameter optimizer mode

The --strategies argument accepts:

- "all"
- a strategy name such as `iron_condor`
- an index such as 7
- a comma-separated combination such as `iron_condor, strangle, 5`

The --strategyomit argument excludes strategies after --strategies filtering is applied.

Thus the final admissible strategy set may be expressed as:

$$S_{active} = S_{included} \setminus S_{omitted}$$

Execution Reality Controls (Phase 5.5)

These arguments govern execution realism.

Argument	Type	Default	Meaning
--no-exec-reality	flag	False	Disable Phase 5.5 execution reality modeling
--exec-latency	float	100.0	Mean order latency in milliseconds
--exec-aggression	float	1.0	Queue aggression from 0 to 1
--exec-seed	int	42	RNG seed for execution reality

These parameters influence whether fills are idealized or subject to latency and stochastic execution assumptions.

v4.3 Multi-Timeframe Inference Controls (Phase 7)

These arguments activate the CondorNet v4.3 inference path.

Argument	Type	Default	Meaning
--use-v43	flag	False	Use CondorNet v4.3 multi-TF inference
--v43-model	str	models/condor_net_v43_run18.pth	Path to v4.3 checkpoint
--v43-data-dir	str	data/Datasetv4/v43	Directory containing v4.3 multi-TF CSVs
--v43-seq-len	int	V43_SEQ_LEN	Inference sequence length

When enabled, the simulator shifts from the legacy CondorBrain v2.2 path to the CondorNet v4.3 multi-timeframe path.

Day-of-Week DTE Entry Caps

These arguments constrain maximum DTE by entry weekday.

Argument	Type	Default	Meaning
--dte-mon	int	5	Max DTE for Monday entries
--dte-tue	int	4	Max DTE for Tuesday entries
--dte-wed	int	3	Max DTE for Wednesday entries
--dte-thu	int	2	Max DTE for Thursday entries
--dte-fri	int	1	Max DTE for Friday entries

These rules exist to limit weekend carry and enforce short-dated structure behavior.

Define weekday-dependent entry cap as:

$$DTE^{max}(d) = \begin{cases} 5 & d = Mon \\ 4 & d = Tue \\ 3 & d = Wed \\ 2 & d = Thu \\ 1 & d = Fri \end{cases}$$

A candidate entry is feasible only if:

$$DTE_{candidate} \leq DTE^{max}(d)$$

14.2 Argument Semantics, Defaults, and Invariants

This section describes what each argument means operationally, what subsystem it affects, and what constraints apply.

Input Data Arguments

`--input` and `--data` both identify the input CSV in the legacy path. Because `--data` is explicitly documented as an alias for `--input`, they should not conflict semantically. If both are supplied, the implementation should resolve them consistently; the documentation should treat them as equivalent entry paths into the same dataset loader.

Invariant:

$$input_path \neq \emptyset$$

for legacy single-file simulation mode.

Failure modes include:

- missing file
 - unreadable CSV
 - missing required columns
 - unsorted or malformed timestamps
-

Model and Ruleset Arguments

`--model` points to the checkpoint used by the legacy neural path.

`--ruleset` points to YAML-based rules.

`--rules-only` disables neural dependency and forces logic-only behavior.

These define the policy source:

$$\pi = \begin{cases} \pi_{rules} & \text{if rules-only} \\ \pi_{model} & \text{if model only} \\ \pi_{hybrid} & \text{if both logic and model layers are active} \end{cases}$$

Failure behavior includes:

- checkpoint file not found
- incompatible checkpoint
- malformed YAML
- rules-only mode requested without compatible rule logic

Hybrid Logic Toggles

--use-fuzzy-sizing activates 11-factor fuzzy sizing logic.
--use-trade-rules enables deterministic entry/exit rules.
--use-diffusion enables diffusion-based parameter refinement.

These toggles alter the decision path and may materially change:

- entry eligibility
- position size
- parameter decoding
- exit behavior

The --limit argument truncates the simulation horizon:

$$N_{bars}^{used} = \min(N_{bars}^{available}, limit)$$

with limit = None implying full dataset usage.

--max-positions constrains the cardinality of the open-position set:

$$|\mathcal{P}_t| \leq \text{max-positions}$$

Valid range:

$$\text{max-positions} \geq 1$$

even though argument parsing itself may accept lower values unless explicitly validated downstream.

Strategy Filter Arguments

--strategies defines the included set.
--strategyomit defines the excluded subset applied afterward.

This means the final eligible strategy universe is not simply chosen once; it is filtered in two stages.

--proffitargets toggles per-strategy dollar targets from STRATEGY_PROFIT_TARGETS. When disabled, the runtime falls back to the standard 50%-of-credit exit behavior.

That policy can be written as:

$$Target_{exit} = \begin{cases} Target_{strategy}(s) & \text{if proffitargets enabled} \\ 0.50 \cdot Credit_{entry} & \text{otherwise} \end{cases}$$

--optimize activates interactive grid-search mode rather than a single standard run.

Failure modes include:

- unknown strategy name
- invalid strategy index
- malformed comma-separated list
- empty active strategy set after omission

Execution Reality Arguments

--no-exec-reality disables the Phase 5.5 realism layer.
Thus execution realism is active unless explicitly disabled.

Define the execution-reality flag as:

$$ExecReality = \neg(\text{no-exec-reality})$$

--exec-latency is the mean order latency in milliseconds and should satisfy:

$$exec_latency \geq 0$$

--exec-aggression is documented as a queue aggression parameter from 0 to 1, with:

- 1.0 corresponding to market-order-like guaranteed fill behavior
- lower values reducing fill probability

Thus valid operational range is:

$$0 \leq \text{exec_aggression} \leq 1$$

--exec-seed fixes the stochastic execution RNG for reproducibility.

Failure modes include:

- negative latency
 - aggression outside the intended interval
 - inconsistent execution assumptions relative to fill model
-

v4.3 Arguments

--use-v43 activates the multi-timeframe CondorNet inference path.

--v43-model specifies the model checkpoint.

--v43-data-dir points to the directory containing v4.3 CSVs.

--v43-seq-len specifies sequence length.

These affect:

- dataset loading pathway
- canonical clock choice
- inference sequence construction
- feature bundle assembly

When v4.3 mode is active, the runtime should be treated as operating on the multi-timeframe bundle rather than the single-file legacy input path.

Invariant:

$$v43_seq_len \geq 1$$

Failure behavior includes:

- checkpoint missing
 - v4.3 directory missing
 - required CSV stack absent
 - sequence length incompatible with available history
-

Day-of-Week DTE Arguments

These parameters impose per-weekday upper bounds on entry DTE. They affect leg discovery and entry feasibility, particularly for short-dated structures.

Operational invariant:

$$dte_mon, dte_tue, dte_wed, dte_thu, dte_fri \geq 1$$

although downstream implementations may permit 0 in some special cases if same-day interpretation is explicit. Based on your help strings, Friday default 1 is intended to mean same-day expiry behavior.

Failure modes include:

- negative DTE caps
- logically inconsistent weekday constraints
- no valid expiries remaining after weekday gating

14.3 Example Runs (Canonical Commands)

Single Legacy Backtest

This runs the legacy single-file path with detailed output and strategy restriction.

```
python condor_brain_backtest_v45.py \  
  --input data/spyohlc/SPY_2025_m1.csv \  
  --model models/condor_brain_v22.pth \  
  --use-trade-rules \  
  --use-fuzzy-sizing \  
  --max-positions 5 \  
  --strategies short_call \  
  --verbose
```

This configuration uses:

- legacy CSV input
- neural model checkpoint
- rule-assisted logic
- fuzzy sizing
- a single active strategy class

Logic-Only Run

This runs the engine without a neural checkpoint.

```
python condor_brain_backtest_v45.py \  
  --input data/spyohlcs/SPY_2025_m1.csv \  
  --ruleset config/trade_rules.yaml \  
  --rules-only \  
  --use-trade-rules \  
  --strategies iron_condor,iron_butterfly \  
  --max-positions 3
```

This configuration is appropriate for deterministic policy testing.

Interactive Optimizer Run

This enables the interactive parameter optimizer mode from the backtester CLI.

```
python condor_brain_backtest_v45.py \  
  --input data/spyohlcs/SPY_2025_m1.csv \  
  --optimize \  
  --strategies all \  
  --max-positions 5 \  
  --profittargets
```

This mode is intended for parameter search rather than a single production-style run.

v4.3 Multi-Timeframe Inference Run

This activates the CondorNet v4.3 path using the multi-timeframe dataset directory.

```
python condor_brain_backtest_v45.py \  
  --use-v43 \  
  --v43-model models/condor_net_v43_run18.pth \  
  --v43-data-dir data/Datasetv4/v43 \  
  --v43-seq-len 10 \  
  --strategies all \  
  --max-positions 5
```

In this mode the runtime uses:

- the v4.3 checkpoint
 - the multi-timeframe dataset stack
 - sequence-based inference
 - portfolio simulation on the v4.3 data layer
-

Execution Reality Enabled Run

Because execution reality is enabled by default unless explicitly disabled, a run with tuned realism controls may be written as:

```
python condor_brain_backtest_v45.py \  
  --input data/spyohlc/SPY_2025_m1.csv \  
  --model models/condor_brain_v22.pth \  
  --exec-latency 100 \  
  --exec-aggression 1.0 \  
  --exec-seed 42 \  
  --strategies short_strangle \  
  --max-positions 2
```

Execution Reality Disabled Run

```
python condor_brain_backtest_v45.py \  
  --input data/spyohlc/SPY_2025_m1.csv \  
  --model models/condor_brain_v22.pth \  
  --no-exec-reality \  
  --strategies iron_condor \  
  --max-positions 5
```

This run is useful for ablation or regression testing against older behavior.

15. Validation Protocol

The CondorNet research platform includes a formal validation protocol designed to ensure that strategy optimization results remain robust and scientifically defensible.

In quantitative finance research, a critical risk is **overfitting**—the selection of parameter configurations that perform well on historical data but fail in unseen market conditions.

To mitigate this risk, CondorNet enforces a strict separation between:

- **optimization data**
- **validation data**

This protocol ensures that strategies are evaluated on data that was not used during parameter selection.

15.1 The Core Rule: Never Optimize and Validate on the Same Slice

The foundational rule of the CondorNet validation framework is:

Optimization and validation must never be performed on the same dataset slice.

Let

$$D$$

represent the available historical dataset.

We partition the dataset into two disjoint subsets:

$$D = D_{train} \cup D_{valid}$$

with

$$D_{train} \cap D_{valid} = \emptyset$$

where

Dataset	Purpose
D_{train}	parameter optimization
D_{valid}	out-of-sample evaluation

The optimizer searches for parameter configurations:

$$\theta^* = \arg \max_{\theta \in \Theta} J(\theta; D_{train})$$

where $J(\theta)$ is the objective function defined in the optimization chapter.

The selected configuration is then evaluated on validation data:

$$Score_{valid} = J(\theta^*; D_{valid})$$

This ensures that the validation score reflects genuine predictive power rather than historical overfitting.

15.2 Temporary 2025-Only Validation Methodology

During early development phases, the available dataset may contain only a single year of data (for example, 2025).

Although multi-year validation is preferable, meaningful validation can still be achieved by dividing the year into multiple temporal segments.

The CondorNet platform therefore employs a **walk-forward validation protocol within the single available year**.

Window Splitting

The first step is to divide the year into contiguous time windows.

Let the dataset be partitioned as:

$$D_{2025} = \{W_1, W_2, \dots, W_n\}$$

where each W_i represents a time window such as one month.

For example:

Window	Period
W_1	January
W_2	February
W_3	March
...	...

Monthly Walk-Forward Validation

The optimizer uses a rolling training window followed by an evaluation window.

For example:

Training Period	Validation Period
Jan–Feb	March
Feb–Mar	April
Mar–Apr	May

Formally, for window i :

$$\theta_i^* = \arg \max_{\theta} J(\theta; W_{i-2}, W_{i-1})$$

and validation is performed on:

$$Score_i = J(\theta_i^*; W_i)$$

This approach simulates how strategies would behave when deployed in real time.

Surface Stability Checks

In addition to walk-forward evaluation, the optimizer examines the **stability of optimization surfaces**.

Recall that optimization produces a mapping:

$$J(\theta)$$

over parameter space.

Stable strategies exhibit:

- broad regions of high performance
- gradual performance gradients
- minimal sensitivity to small parameter changes

In contrast, unstable strategies exhibit:

- sharp isolated peaks
- extreme parameter sensitivity
- inconsistent performance across adjacent parameter values

Researchers therefore prioritize parameter regions that satisfy:

$$\frac{\partial J}{\partial \theta} \approx 0$$

across a local neighborhood.

Such regions indicate robust configurations that are less likely to overfit historical data.

15.3 Future Multi-Year Regime Validation Plan (2022–2025)

Once multi-year datasets become available, the validation framework will expand to incorporate regime-aware evaluation.

The planned dataset will include several years of historical market data:

$$D = D_{2022} \cup D_{2023} \cup D_{2024} \cup D_{2025}$$

This allows strategies to be tested across different market regimes.

Regime Diversity

Different years may represent distinct volatility regimes.

Examples include:

Regime	Characteristics
low volatility	narrow trading ranges
high volatility	large price swings
trending markets	directional momentum
mean-reverting markets	oscillating behavior

Strategies must demonstrate robustness across multiple regimes.

Multi-Year Training and Validation

One possible validation configuration is:

Training	Validation
2022–2023	2024
2022–2024	2025

Formally:

$$\theta^{*} = \arg \max_{\theta} J(\theta; D_{2022:2023})$$

and

$$Score_{valid} = J(\theta^{*}; D_{2024})$$

This allows evaluation on previously unseen market conditions.

Rolling Regime Validation

Another approach is rolling multi-year validation.

Example sequence:

Train	Test
2022	2023
2022–2023	2024
2023–2024	2025

This approach simulates how strategies would evolve through time.

Appendices

Appendix A — File Index (Current)

This appendix provides a structural index of the CondorNet v4.5 codebase. It enumerates the primary modules, their exported functions and classes, and how they participate in the system architecture.

The purpose of this appendix is to allow engineers, reviewers, and auditors to quickly locate the implementation of each subsystem described in the main documentation.

A.1 condor_brain_backtest_v45.py — Function and Class Index

The file **condor_brain_backtest_v45.py** implements the primary simulation engine and serves as the runtime entry point for the backtester.

This module contains the following major responsibilities:

- simulation loop
- trade lifecycle management
- execution modeling
- strategy evaluation
- event logging
- dataset generation

The script also defines the **command-line interface and runtime configuration**.

A.1.1 Runtime Entry Point

main()

Primary CLI entrypoint responsible for:

1. parsing command-line arguments
2. loading model and datasets
3. initializing simulation state
4. invoking the backtest loop
5. writing reports

The CLI arguments include parameters for:

- dataset selection

- model loading
- execution modeling
- optimization
- strategy filtering
- DTE constraints
- inference configuration

A.1.2 Core Simulation Functions

run_backtest(...)

Central simulation driver responsible for executing the bar loop.

Responsibilities include:

- market state construction
- open position valuation
- exit condition evaluation
- entry condition evaluation
- execution simulation
- trade lifecycle recording

Outputs include:

- equity curve
- closed trades
- event logs

select_best_template(...)

Chooses the most appropriate strategy template given:

- market features
- model predictions
- predicate filters

Returns:

```
template_id  
template_config
```

find_best_legs(...)

Searches the option chain for candidate contracts that satisfy:

- delta targets
- strike offsets
- liquidity constraints
- expiration rules

Returns a set of option legs that satisfy template structure.

`validate_leg_liquidity(...)`

Ensures that candidate option contracts satisfy liquidity rules such as:

- minimum open interest
- acceptable bid/ask spread
- valid pricing

`synthesize_chain_prices(...)`

Ensures the options chain has consistent pricing fields:

mid
close
spread_ratio

This function normalizes chain pricing for downstream execution modeling.

A.1.3 Trade Lifecycle Functions

`open_trade(...)`

Creates a new trade object when entry conditions pass.

Responsibilities include:

- constructing leg structures
- computing margin requirements
- recording entry metadata

`close_trade(...)`

Closes an existing trade and records:

```
realized PnL  
exit reason  
hold duration
```

update_open_positions(...)

Evaluates all open positions at each bar to compute:

- unrealized PnL
 - updated margin usage
 - expiration handling
-

A.1.4 Execution Reality Functions

simulate_entry_fill(...)

Models entry order execution including:

- spread crossing
 - slippage
 - atomic multi-leg fills
-

simulate_exit_fill(...)

Models closing transactions using realistic bid/ask pricing.

compute_slippage(...)

Calculates expected fill deviation from mid price.

A.1.5 Reporting Functions

write_trade_log(...)

Writes completed trades to:

```
reports/trades.csv
```

```
write_equity_curve(...)
```

Exports the portfolio equity time series to:

```
reports/equity_curve.csv
```

```
emit_bar_trace(...)
```

Records decision trace events for debugging and dataset generation.

Output file:

```
reports/bar_trace.jsonl
```

A.2 backtest_v45_data.py — Function and Class Index

The module **backtest_v45_data.py** provides the complete multi-timeframe data ingestion and preprocessing layer used by the backtester.

Its responsibilities include:

- dataset loading
- timeframe alignment
- normalization
- options chain ingestion
- batch inference preparation

backtest_v45_data

A.2.1 Core Data Structure

MultiTFDataBundle

A dataclass containing all preprocessed data required by the simulation.

Key fields include:

Field	Description
m1_feat	normalized M1 features
m5_feat	normalized M5 features
m15_feat	normalized M15 features
h1_feat	normalized H1 features
m1_piv	pivot indicators

m5_piv	pivot indicators
chain_grid_by_date	model inference grids
chain_df_by_date	raw option chain
pos_state	position-state features
m5_timestamps	canonical clock
m5_spot	underlying price

The bundle ensures that all datasets share the same primary index:

N = number of M5 bars

A.2.2 Dataset Loader

`load_multi_tf_bundle(...)`

Loads and preprocesses the entire dataset stack.

Responsibilities:

1. load M5 canonical dataset
2. load other timeframes
3. align timestamps
4. normalize features
5. load options chain
6. construct chain grids

Returns:

MultiTFDataBundle

A.2.3 Alignment Functions

`_align_to_m5(...)`

Maps all timeframe data to the canonical M5 clock using timestamp matching.

Alignment rule:

```
idx = searchsorted(other_ts, m5_ts, side='right') - 1
```

A.2.4 Normalization Functions

`_robust_zscore(...)`

Applies robust z-score normalization using:

$$z = \frac{x - \text{median}}{1.4826 \cdot MAD}$$

A.2.5 Options Chain Processing

`_build_chain_grid(...)`

Constructs a fixed-size tensor representation of the option chain.

Output shape:

```
[N_contracts, 10]
```

Features include:

- moneyness
- DTE
- implied volatility
- delta
- gamma
- theta
- vega
- bid
- ask
- normalized open interest

A.2.6 Inference Pipeline

`run_v43_batch_inference(...)`

Runs CondorNet v4.3 inference across the entire dataset.

Returns arrays:

```
entry_signal  
exit_signal  
pop  
ev  
position_size  
strategy_idx  
abstain
```

A.2.7 Chain Normalization

`normalize_v43_chain(...)`

Standardizes options chain column naming conventions for compatibility with the execution engine.

A.2.8 Model Loader

`load_v43_model(...)`

Loads a trained CondorNet v4.3 model checkpoint.

Handles:

- architecture reconstruction
- parameter loading
- device placement

A.3 strategy_optimizer.py — Function Index

The file **strategy_optimizer.py** implements the parameter sweep optimization system.

It performs grid-search optimization over strategy parameters and ranks configurations by performance.

strategy_optimizer

A.3.1 Primary Entry Function

`run_optimizer(...)`

Runs the full optimization process.

Responsibilities:

1. identify strategy template to optimize
2. prompt user for parameter ranges
3. generate parameter grid
4. run backtests for each configuration
5. compute performance metrics
6. rank results
7. allow best configuration to be applied

A.3.2 Parameter Grid Generation

Uses Cartesian product over parameter ranges:

```
combos = itertools.product(*all_vals)
```

This produces a parameter search space.

A.3.3 Performance Metrics Computation

For each configuration the optimizer computes:

Metric	Description
net_pnl	total profit
net_pct	percent return
max_dd	maximum drawdown
np_dd	net profit / drawdown
win rate	proportion of profitable trades
Sharpe	return volatility ratio
expectancy	average profit per trade
profit factor	gross win / gross loss

A.3.4 Result Storage

Optimization results are written to:

```
reports/optimizer_<template>_<timestamp>.csv
```

A.3.5 Template Update

After selecting the best result, the optimizer automatically updates the strategy template configuration file.

This modifies the `CONFIG` dictionary within:

```
strategies/<template>.py
```

A.4 strategies/ — Template Inventory and Mapping

The `strategies/` directory contains the complete catalog of strategy templates.

Each file defines a strategy via a `CONFIG` dictionary describing:

- leg structure
- strike targeting
- DTE rules
- margin model
- risk parameters

A.4.1 Template Structure

Each strategy module contains:

```
CONFIG = {  
    "template_id": "...",  
    "legs": [...],  
    "profit_target": ...,  
    "stop_loss": ...,  
    "max_positions": ...  
}
```

A.4.2 Template Loader

The loader:

strategies/___init___ .py

performs:

- directory scanning
- exclusion of `*.py` files
- CONFIG extraction
- validation of required keys

A.4.3 Strategy Class Mapping

During model inference the neural network outputs:

strategy_idx

which maps to strategy classes.

Example mapping:

Index	Strategy
0	short call
1	short put

2	vertical_spread
3	iron_condor
4	iron_butterfly
5	strangle
6	straddle
7	calendar
8	diagonal
9	abstain

This mapping allows the policy model to select a **strategy class**, which is then resolved to a specific template in the strategy catalog.

Appendix B — Formal Mathematics

This appendix documents the mathematical definitions used in the CondorNet backtesting and strategy evaluation framework.

The formulas govern:

- feature normalization
- option position valuation
- margin estimation
- risk control
- optimizer ranking

All notation follows standard derivatives-trading conventions.

B.1 Robust Normalization Derivation

(MAD scaling constant 1.4826)

Financial datasets frequently contain **heavy-tailed distributions** and **extreme outliers**. Standard z-score normalization

$$z_i = \frac{x_i - \mu}{\sigma}$$

is unstable when large outliers distort the sample mean and variance.

For this reason CondorNet uses **Median Absolute Deviation (MAD)** normalization.

Median

Given a feature vector

$$X = \{x_1, x_2, \dots, x_n\}$$

the sample median is

$$\tilde{x} = \text{median}(X)$$

Median Absolute Deviation

The MAD is defined as

$$\text{MAD} = \text{median}(|x_i - \tilde{x}|)$$

This statistic measures dispersion around the median.

Relationship to Gaussian Standard Deviation

For a normally distributed variable:

$$X \sim N(\mu, \sigma)$$

the expected value of the MAD satisfies

$$E[\text{MAD}] = \sigma \cdot \Phi^{-1}(0.75)$$

where

$$\Phi^{-1}(0.75) \approx 0.67449$$

Thus

$$\sigma \approx \frac{\text{MAD}}{0.67449}$$

and

$$\frac{1}{0.67449} \approx 1.4826$$

Robust Z-Score

Therefore CondorNet computes normalized features as

$$z_i = \frac{x_i - \tilde{x}}{1.4826 \cdot \text{MAD}}$$

Practical Implementation

To prevent exploding features the system applies clipping

$$z_i \leftarrow \text{clip}(z_i, -10, 10)$$

This ensures numerical stability during model inference.

B.2 Margin / Collateral Formulas by Structure Class

The backtester estimates required capital for each position based on its structure.

Let

$$\begin{aligned} Q &= \text{number of contracts} \\ M &= 100 \end{aligned}$$

the option multiplier.

Naked Short Option

For a short call or put without protection:

$$\text{Collateral} = Q \cdot M \cdot S$$

where

$$S = \text{underlying price}$$

This is a conservative approximation of broker margin rules.

Vertical Spread

For a vertical credit spread:

$$\text{Collateral} = Q \cdot M \cdot |K_{long} - K_{short}|$$

where

$$K_{long}, K_{short}$$

are strike prices.

Iron Condor

An iron condor is two spreads.

Required collateral is the larger wing width.

$$\text{Collateral} = Q \cdot M \cdot \max(|K_{call,long} - K_{call,short}|, |K_{put,long} - K_{put,short}|)$$

Straddle / Strangle

If naked:

$$\text{Collateral} = Q \cdot M \cdot S$$

If defined-risk wings exist, the calculation reduces to spread width.

B.3 Credit, Maximum Loss, and PnL

Net Credit

Let each leg have entry price p_i .

$$\text{Credit} = \sum_{i \in \text{short}} p_i - \sum_{i \in \text{long}} p_i$$

Total position credit:

$$C = Q \cdot M \cdot \text{Credit}$$

Maximum Loss

Defined Risk Structures

For spreads:

$$\text{MaxLoss} = Q \cdot M \cdot (W - \text{Credit})$$

where

$$W = |K_{\text{long}} - K_{\text{short}}|$$

Naked Short Option

For naked short calls:

$$\text{MaxLoss} \rightarrow \infty$$

For naked puts:

$$\text{MaxLoss} = Q \cdot M \cdot K$$

where

$$K = \text{strike price}$$

Mark-to-Market PnL

At time t :

$$\text{PnL}_t = Q \cdot M(C - V_t)$$

where

$$V_t = \text{current mark value of the position}$$

B.4 Stop Loss and Profit Target Definitions

Risk controls are defined as functions of maximum risk.

Let

$$L_{max} = \text{maximum possible loss}$$

Profit Target

Profit target fraction:

$$\alpha \in (0,1)$$

Close the position when

$$\text{PnL}_t \geq \alpha \cdot C$$

Typical values:

$$\alpha = 0.50$$

Stop Loss

Stop loss fraction:

$$\beta > 1$$

Close when

$$\text{PnL}_t \leq -\beta \cdot C$$

Example:

$$\beta = 2$$

meaning loss equals twice collected credit.

Time Stop

Positions may also close at

$$\text{DTE} \leq D_{min}$$

to avoid expiration risk.

B.5 NP/DD Ranking Function

Strategy optimization ranks parameter sets using the ratio:

$$\text{Score} = \frac{\text{Net Profit}}{\text{Maximum Drawdown}}$$

where

$$\text{Net Profit} = \sum_{i=1}^N \text{PnL}_i$$

and

$$\text{Maximum Drawdown} = \max_t \left(\frac{E_{peak} - E_t}{E_{peak}} \right)$$

with E_t the equity curve.

Interpretation

The metric measures **profit generated per unit of downside risk**.

Higher values indicate better capital efficiency.

ALTERNATIVE RANKING METRICS

Sharpe Ratio

$$\text{Sharpe} = \frac{E[R]}{\sigma_R}$$

Sortino Ratio

$$\text{Sortino} = \frac{E[R]}{\sigma_{\text{downside}}}$$

Profit Factor

$$\text{PF} = \frac{\text{Gross Profit}}{\text{Gross Loss}}$$

Expectancy

$$E = P_{\text{win}} \cdot \bar{W} - P_{\text{loss}} \cdot \bar{L}$$

SUMMARY

The CondorNet backtester relies on mathematically defined components for:

- robust feature normalization
- collateral estimation
- position valuation
- automated risk controls
- strategy ranking

These formulas ensure that simulation results are **consistent, reproducible, and financially interpretable**.

Appendix C — Data Schemas

This appendix defines the primary structured records emitted by the CondorNet v4.5 platform.

These schemas are designed to support:

- auditability
- lifecycle reconstruction
- optimizer analysis
- machine learning dataset generation

All schemas should be treated as **logical contracts** even if the concrete CSV / JSONL / Parquet columns evolve slightly during implementation.

C.1 OPEN Event Schema

The OPEN event records the creation of a new position. Each OPEN event must uniquely identify the newly created trade and preserve enough information to reconstruct:

- why the trade was opened
- what structure was opened
- what capital was committed
- how the legs were priced

Logical schema

Field	Type	Description
event_type	string	Must equal OPEN
timestamp	datetime	Entry timestamp
trade_id	string	Unique position identifier
strategy	string	Strategy template ID
strategy_class	string	Higher-level strategy class
template_id	string	Exact template used
action	string	ENTER
entry_credit	float	Net credit or debit at entry
contracts	int	Number of contracts
margin	float	Required collateral
max_loss	float	Estimated maximum loss
dte_entry	float	Days to expiration at entry
spot	float	Underlying spot price at entry
legs	object / JSON	Full leg structure
fill_details	object / JSON	Bid/ask/slippage/fill diagnostics
gate_trace	object / JSON	Entry gate pass/fail decomposition
policy_scores	object / JSON	Entry logit / POP / EV / confidence
portfolio_deployed_before	float	Capital deployed before entry
portfolio_deployed_after	float	Capital deployed after entry

Required invariants

event_type = OPEN
trade_id ≠ ∅
contracts ≥ 1
margin ≥ 0

C.2 CLOSE Event Schema

The CLOSE event records the full closure of an existing trade.

It must contain enough information to reconstruct:

- why the position exited
- how long it was held
- what PnL was realized
- whether the exit was policy-driven, risk-driven, or forced

Logical schema

Field	Type	Description
event_type	string	Must equal CLOSE
timestamp	datetime	Exit timestamp
trade_id	string	Position identifier
action	string	CLOSE
exit_reason	string	Profit target, stop, expiry, Friday closeout, etc.
heldBars	int	Number of bars held
heldDays	float	Holding duration in days
realized_pnl	float	Final realized profit or loss
pnl_pct_of_risk	float	PnL as fraction of risk or max loss
exit_debit	float	Debit paid to close, if applicable
fill_valid	bool	Whether close fill used primary execution model
fill_details	object / JSON	Exit pricing diagnostics
equity_after	float	Cash equity after close
max_dd_trade	float	Max adverse excursion for that trade

Required invariants

$$\begin{aligned} &event_type = \text{CLOSE} \\ &trade_id \in \{trade_id \text{ values previously opened}\} \\ &held_bars \geq 0 \end{aligned}$$

A CLOSE event must correspond to exactly one prior OPEN event for the same `trade_id`.

C.3 StateActionRecord Schema

The StateActionRecord is the primary machine-learning training row.

It represents one timestep of decision data in the form:

$$(s_t, a_t, r_t, s_{t+1})$$

This record may be emitted per bar, per position, or per candidate decision depending on the dataset mode.

Logical schema

Field	Type	Description
timestamp	datetime	Decision timestamp
trade_id	string / nullable	Associated trade, if any
state_id	string	Unique row identifier
market_state	vector / JSON	Multi-TF market features
portfolio_state	vector / JSON	Portfolio summary features
strategy_context	vector / JSON	Template ID, params, cooldown, etc.
action	string	ENTER / HOLD / EXIT / MODIFY / ADD
action_param	string / JSON	Strategy name, exit reason, modify type, add size
reward_immediate	float	Immediate reward
reward_terminal	float	Terminal reward if event is terminal
reward_total	float	Combined shaped reward
constraint_penalty	float	Penalty applied for undesired behavior
next_state_ref	string / nullable	Reference to next row or state ID
is_terminal	bool	Whether the transition terminates the trade
event_ref	string / nullable	Associated OPEN/CLOSE/MODIFY/ADD event row

Required invariants

$$\begin{aligned}
 &action \in \{\text{ENTER}, \text{HOLD}, \text{EXIT}, \text{MODIFY}, \text{ADD}\} \\
 &reward_total = reward_immediate + reward_terminal - constraint_penalty
 \end{aligned}$$

When `is_terminal = True`, the row should correspond to a terminal lifecycle action, usually `CLOSE` or a terminal `EXIT` transition.

C.4 OptimizationRunRow Schema

The `OptimizationRunRow` stores the result of one optimizer evaluation over one parameter vector.

Each row corresponds to one full backtest run under one configuration.

Logical schema

Field	Type	Description
run_id	string	Unique optimizer run identifier
timestamp	datetime	Time row was written
strategy	string	Template ID being optimized
timeframe	string	Runtime timeframe or dataset mode
stop_loss_dollar	float / nullable	Tested parameter
profit_target	float / nullable	Tested parameter
max_positions	int	Tested parameter
cooldownBars	int	Tested parameter
stop_loss_mult	float	Tested parameter
max_contracts	int	Tested parameter
net_pnl	float	Net profit
net_pct	float	Net return percentage
max_dd	float	Maximum drawdown
np_dd	float	Net-profit / drawdown ranking score
wins	int	Number of winning trades
losses	int	Number of losing trades
wr	float	Win rate
sharpe	float	Sharpe ratio
expectancy	float	Expected profit per trade
pf	float	Profit factor
max_consec_l	int	Maximum consecutive losses
total_trades	int	Total completed trades
gate_fail_counts	object / JSON	Optional gate rejection counters
capital_block_count	int / nullable	Number of capital-gate rejections
chain_fail_count	int / nullable	Number of chain-integrity failures

Required invariants

$$\begin{aligned}total_trades &= wins + losses \\max_dd &\geq 0\end{aligned}$$

If drawdown is nonzero, the primary optimizer score is

$$np_dd = \frac{net_pct}{max_dd}$$

or the implementation's equivalent normalized form.

Appendix D — Strategy CONFIG Contract

The Strategy CONFIG contract defines the formal schema that every strategy template must implement.

Each strategy template located in `strategies/*.py` exposes a single CONFIG dictionary describing its behavior.

This contract ensures:

- deterministic strategy loading
- safe optimizer modification
- template validation
- compatibility with the backtester engine

D.1 Required Keys

Every strategy CONFIG must contain the following fields.

Key	Type	Description
<code>template_id</code>	string	Unique identifier for the template
<code>strategy_class</code>	string	High-level category (short call, condor, etc.)
<code>legs</code>	list	Leg definitions describing option structure
<code>entry_credit_min</code>	float	Minimum acceptable entry credit
<code>profit_target</code>	float	Fraction of credit to capture
<code>stop_loss_mult</code>	float	Stop loss multiplier vs entry credit
<code>max_positions</code>	int	Maximum simultaneous positions
<code>cooldownBars</code>	int	Minimum bars between entries
<code>margin_model</code>	string	Method for collateral calculation
<code>dte_min</code>	float	Minimum days to expiration
<code>dte_max</code>	float	Maximum days to expiration

Leg Definition Structure

Each element of `legs` must define the structure of a position leg.

Required fields per leg:

Key	Description
type	call or put
side	long or short
delta_target	target option delta
qty	number of contracts

Example:

```
{  
  "type": "call",  
  "side": "short",  
  "delta_target": 0.30,  
  "qty": 1  
}
```

D.2 Optional Keys and Defaults

Some parameters may be omitted from a CONFIG file.
When missing, the engine merges them with global defaults from:

strategies/_defaults.py

Optional parameters include:

Key	Default	Description
entry_ev_min	0.0	Minimum expected value
pop_min	0.50	Minimum probability of profit
max_contracts	1	Maximum contracts per trade
spread_ratio_max	0.30	Liquidity filter
min_open_interest	10	OI filter
min_volume	1	Volume filter
capital_fraction	0.10	Max capital per trade
friday_closeout	true	Avoid weekend risk
dte_closeout	1	Minimum DTE before forced close
allow_add	false	Allow position scaling

D.3 Allowed Ranges and Validation Rules

The loader performs validation before registering templates.

Profit Target

$$0 < profit_target \leq 1$$

Typical values:

0.30 – 0.70

Stop Loss Multiplier

$$stop_loss_mult > 1$$

Typical values:

1.5 – 3.0

Delta Targets

$$0 < |\Delta| < 1$$

Typical ranges:

Strategy	Delta
Short call	0.25–0.35
Short put	–0.25––0.35
Condor wings	0.10–0.20

DTE Bounds

$$0 < dte_min < dte_max$$

Typical ranges:

30 – 60 days

Liquidity Constraints

Valid contracts must satisfy:

$$\begin{aligned}OI &\geq OI_{min} \\ Volume &\geq Volume_{min} \\ \frac{ask - bid}{mid} &\leq spread_ratio_{max}\end{aligned}$$

D.4 Example — Annotated `short_call` CONFIG

Below is a complete example of a minimal strategy template.

```
CONFIG = {  
  
    # Unique identifier  
    "template_id": "short_call",  
  
    # Strategy classification  
    "strategy_class": "short_call",  
  
    # Option leg structure  
    "legs": [  
        {  
            "type": "call",  
            "side": "short",  
            "delta_target": 0.30,  
            "qty": 1  
        }  
    ],  
  
    # Entry requirements  
    "entry_credit_min": 0.25,  
  
    # Profit target (fraction of credit)  
    "profit_target": 0.50,  
  
    # Stop loss multiplier  
    "stop_loss_mult": 2.0,  
  
    # Position limits  
    "max_positions": 3,  
  
    # Entry cooldown  
    "cooldownBars": 12,  
  
    # Collateral model  
    "margin_model": "naked_pct_spot",  
  
    # Expiration constraints  
    "dte_min": 30,  
    "dte_max": 60  
}
```

How the Engine Uses This CONFIG

1. Template loader registers the strategy.
2. Backtester evaluates entry predicates.
3. `find_best_legs()` searches the chain for contracts matching delta targets.
4. The margin model estimates required collateral.
5. The trade lifecycle follows the defined stop and target rules.

Appendix E — Reproducibility & Audit Checklist

This appendix defines the operational requirements necessary to guarantee that every CondorNet backtest, optimization run, or dataset generation process is **fully reproducible and auditable**.

The intent is that any reviewer, auditor, or developer can reproduce a run **bit-for-bit** given the proper artifacts and configuration.

This section covers:

- determinism requirements
- artifact preservation requirements
- the minimal audit bundle that must accompany every run

E.1 Determinism Checklist

Deterministic execution ensures that identical inputs always produce identical outputs.

For the CondorNet research platform, determinism must be guaranteed across:

- dataset loading
- inference
- strategy selection
- execution simulation
- optimizer runs

The following checklist defines required controls.

1. Random Seed Control

All stochastic components must be initialized with fixed seeds.

Required seeds:

Component	Variable
Python random	PYTHONHASHSEED
NumPy	np.random.seed()
PyTorch CPU	torch.manual_seed()
PyTorch GPU	torch.cuda.manual_seed_all()

Example initialization:

```
import random
import numpy as np
import torch

SEED = 1337

random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)
```

2. Deterministic Library Modes

Deep learning frameworks may introduce nondeterminism through parallel kernels.

Recommended settings:

```
torch.use_deterministic_algorithms(True)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
```

3. Deterministic Template Ordering

Strategy template discovery must always produce a consistent order.

Enforce lexicographic ordering:

```
sorted(strategies/*.py)
```

This prevents template index drift across runs.

4. Deterministic Chain Selection

Option selection routines must follow deterministic ordering rules.

Example rule:

1. Sort by

- delta distance
 - open interest
 - strike distance
2. Select the first valid candidate.

No random sampling is permitted.

5. Fixed Dataset Snapshots

Backtests must run against immutable datasets.

Dataset integrity must be verified using checksums.

Example:

```
SHA256(m5_dataset_v43_final.csv)
SHA256(options_2025_v43.csv)
```

6. Configuration Freezing

Every run must record the exact configuration used.

Required configuration artifacts:

- CLI arguments
- strategy CONFIG
- optimizer parameter ranges
- margin model rules

7. Environment Capture

Execution environment must be documented.

Required information:

Parameter	Example
Python version	3.12
PyTorch version	2.x
CUDA version	12.x

OS	Linux
----	-------

E.2 Artifact Checklist

Each backtest run must produce a set of artifacts sufficient to reconstruct the experiment.

The following files must be retained.

Core Simulation Artifacts

Artifact	Purpose
trades.csv	full trade lifecycle
equity_curve.csv	portfolio equity over time
bar_trace.jsonl	per-bar decision trace

Optimization Artifacts

Artifact	Purpose
optimizer_results.csv	full parameter sweep results
optimizer_config.json	tested parameter ranges

Dataset Artifacts

Artifact	Purpose
trajectories.parquet	RL-style state-action dataset
event_log.jsonl	event-level lifecycle record

Model Artifacts

If CondorNet inference is used:

Artifact	Purpose
condor_net_v43.pth	model checkpoint
model_config.json	architecture parameters

Configuration Artifacts

Artifact	Purpose
run_config.json	full CLI argument snapshot
strategy_catalog_snapshot.json	template registry snapshot

E.3 Minimal Audit Bundle

To allow external reviewers to verify results, every run must produce a **minimal audit bundle**.

This bundle is the smallest collection of files required to fully reproduce the run.

Required Bundle Contents

File	Description
run_config.json	complete CLI argument set
strategy_catalog_snapshot.json	strategy template registry
dataset_checksums.json	hashes of all datasets
trades.csv	executed trades
equity_curve.csv	equity time series
bar_trace.jsonl	per-bar decision trace

Optional but Recommended

File	Description
optimizer_results.csv	optimizer output
trajectories.parquet	ML training dataset
model_checkpoint.pth	neural inference model

Audit Bundle Directory Layout

Recommended directory structure:

```
audit_bundle/  
├── config/  
│   ├── run_config.json  
│   └── strategy_catalog_snapshot.json  
├── data_checksums/  
│   └── dataset_checksums.json  
├── results/  
│   ├── trades.csv  
│   └── equity_curve.csv  
└── traces/
```

```
|   bar_trace.jsonl  
|  
└ optional/  
  optimizer_results.csv  
  trajectories.parquet  
  condor_net_v43.pth
```

Reproduction Procedure

To reproduce a run:

1. Verify dataset checksums.
2. Load the run configuration.
3. Restore strategy catalog snapshot.
4. Execute the backtester using the saved CLI arguments.
5. Compare produced artifacts with stored artifacts.

Reproducibility is verified if:

$$\text{EquityCurve}_{new} = \text{EquityCurve}_{original}$$

and

$$\text{Trades}_{new} = \text{Trades}_{original}$$